Universidad Distrital Francisco José de Caldas
Ingeniería Electrónica · Análisis y Diseño de Microprocesadores

# TALLER 5

# Microprocesador de propósito específico: determinante por el algoritmo de Bareiss con salida BCD (algoritmo XS3)

Caja negra, cajas grises, pseudoalgoritmo, diagramas ASM y código VHDL

Herramienta: Xilinx ISE 14.7 (XST, ISim) · Tarjeta: Digilent Nexys A7-100T (XC7A100T-1CSG324C)

**Nota para el equipo.** Este documento explica un diseño de referencia. El enunciado advierte que los diseños parecidos entre equipos no se califican, así que revisen cada estado y cada bloque, entiendan por qué está ahí y adáptenlo antes de entregarlo. Las conclusiones del informe son responsabilidad del equipo.

# Contenido


**1. Resumen del diseño** **3**
- Reglas de descripción . . . 3
- Requisitos del enunciado y de la clase . . . 3
- Archivos del repositorio . . . 4

**2. Repaso del algoritmo de Bareiss** **4**
- Ejemplo 3×3 . . . 4
- Pivote cero . . . 4

**3. Caja negra** **6**
- Caja negra de la tarjeta . . . 6

**4. Cajas grises** **7**
- Cajas grises de los bloques con ASM propio . . . 8
- Tope de la tarjeta . . . 8

**5. Pseudoalgoritmo con el hardware disponible** **8**

**6. Diagrama ASM principal** **10**
- Convenciones . . . 10
- Vista general . . . 10
- Hoja 1: espera y carga de la matriz (s0 a s3) . . . 11
- Hoja 2: pivote, búsqueda de fila, intercambio y banco (s4 a s9) . . . 12
- Hoja 3: productos, resta e inicio de la división (s10 a s15) . . . 13
- Hoja 4: espera de la división, escritura y avance (s16 a s18) . . . 14
- Hoja 5: conversión XS3 y final (s19 a s21) . . . 15
- Tabla de estados . . . 16

**7. ASM del bloque de división** **17**

**8. ASM del bloque XS3** **17**
- Traza del XS3 para 362 . . . 19

**9. Señales del ASM principal** **20**
- 9.1 Señales de control por estado (ASM → bloques) . . . 20
- 9.2 Señales de estado (bloques → ASM) . . . 20
- 9.3 Registros del ASM principal y anchos . . . 21

**10. Interfaz con la tarjeta** **22**
- Divisor de frecuencia . . . 22
- Antirrebote . . . 22
- Sincronizador y display . . . 22

**11. Tiempo de ejecución** **22**

**12. Implementación en ISE 14.7 y la Nexys A7** **22**
- 12.1 Proyecto . . . 22
- 12.2 Cambiar la matriz (día de la evaluación) . . . 22
- 12.3 Operación en la tarjeta . . . 23
- 12.4 Matrices de prueba . . . 23
- 12.5 Pines (src/nexys_a7.ucf) . . . 24

**13. Simulación** **24**

**14. Limitaciones y verificación pendiente** **24**

**Anexo A. Código VHDL** **26**

# 1. Resumen del diseño

El procesador calcula el determinante de la matriz guardada en su ROM (de 2×2 a 5×5) con el algoritmo de Bareiss, guarda los pivotes en un banco de registros y entrega el determinante y el pivote que indica `dir_p` en BCD, convertidos con el algoritmo XS3. La ROM hace parte de la arquitectura: para calcular el determinante de otra matriz se cambia el archivo de la ROM y se vuelve a programar la FPGA.

Siguiendo la clase del 9 de noviembre, el diseño es un híbrido entre un camino de datos estructural y un ASM de alto nivel. Los bloques principales (ROM, RAM, multiplicador, sumador/restador, división, XS3 y banco de pivotes) están descritos aparte e interconectados en la entidad `procesador`. El ASM principal (`asm_bareiss`, 22 estados) se encarga del direccionamiento y de distribuir las señales a esos bloques. La división y el XS3 tienen cada uno su propia unidad de control y su propio camino de datos.

## Reglas de descripción

| Regla | Cómo se cumple |
|---|---|
| Solo std_logic, excepto la RAM | Todas las señales, puertos y constantes son std_logic o std_logic_vector. La RAM sigue el modelo del curso (`type mem is array … of std_logic_vector` y `conv_integer` en la línea de dirección); es el único lugar del diseño donde aparecen. |
| Sin integer, signed ni unsigned | No hay `integer`, `natural`, `signed`, `unsigned`, `boolean`, tipos enumerados ni genéricos. Los estados son constantes `std_logic_vector`; el `case st is / when s0 =>` se escribe igual que en los ejemplos de clase. |
| Sin variables ni descripciones tipo software | No hay `variable` ni lazos `for`/`while`. Las repeticiones del algoritmo las hacen los ASM, un paso por ciclo de reloj, con contadores. |
| Sintetizable y con esquemático RTL claro | Un archivo por bloque y entidades estructurales (`procesador`, `det_bareiss`) que solo conectan bloques. La síntesis genérica con GHDL termina sin errores ni latches. |
| Un solo multiplicador y un solo sumador/restador | Entidades `multiplicador` y `sumador_restador`, compartidas por los estados del ASM principal mediante multiplexores. El bloque de división tiene su restador propio (es el «bloque de división» del enunciado) y el XS3 su sumador de 4 bits (+3). |
| Solo para simulación | Los testbench usan variables, integer, real, string y wait; no se sintetizan. |

## Requisitos del enunciado y de la clase

| Requisito | Cómo lo cumple el diseño |
|---|---|
| Señal de inicio y señal de final | Entradas y salidas del procesador; `final = '1'` en el estado s21. |
| Matriz en ROM: 00h = n, luego los a(i,j) como en la Figura 1 | Entidad `rom_matriz` (src/rom_matriz.vhd): dirección `dira` de 5 bits y dato a de 6 bits en complemento a 2. Elementos por columnas: a11, a21, …, an1, a12, … |
| La ROM hace parte de la arquitectura | La ROM está dentro del procesador; no se ingresa por puertos. |
| Multiplicador, sumador/restador y división por restas sucesivas | Entidades `multiplicador`, `sumador_restador` y `bloque_division`. |
| Memoria RAM para las iteraciones | Entidad `RAM_memory` (modelo del curso), 64 × 32 bits. |
| Pivotes en un banco de registros, visibles con los switches | Entidad `banco_pivotes`. dir_p = k (SW2..SW0) elige el pivote; el último pivote (k = n) es el determinante. |
| Determinante en los displays, en decimal | Salida `det` en BCD; con dir_p = 0 el display muestra det. |
| Bloque XS3 con su unidad de control y su camino de datos | Entidad `xs3`: ASM de 6 estados, un estado por cada paso del documento XS3. |

| Requisito | Cómo lo cumple el diseño |
|---|---|
| Antirrebote solo para el botón de inicio | Entidad `antirrebote` (src/antirrebote.vhd). |

### Archivos del repositorio

| Archivo | Contenido |
|---|---|
| src/det_bareiss.vhd | Tope para la Nexys A7 (estructural). |
| src/procesador.vhd | Procesador: une el ASM con sus bloques (estructural). |
| src/asm_bareiss.vhd | ASM principal (s0 a s21). |
| src/bloque_division.vhd, src/xs3.vhd | Bloques con ASM propio: división por restas sucesivas y XS3. |
| src/rom_matriz.vhd, src/ram.vhd, src/banco_pivotes.vhd | ROM con la matriz, RAM (modelo del curso), banco de pivotes. |
| src/multiplicador.vhd, src/sumador_restador.vhd | Unidades aritméticas. |
| src/divisor_frecuencia.vhd, src/antirrebote.vhd, src/sincronizador.vhd, src/display_8dig.vhd | Interfaz con la tarjeta. |
| src/nexys_a7.ucf | Pines y restricción de reloj (100 MHz). |
| sim/tb_procesador.vhd, sim/pkg_prueba.vhd | Testbench autoverificado del procesador: sirve para cualquier ROM. |
| sim/tb_det_bareiss.vhd | Testbench de la tarjeta: botón con rebotes y lectura del display. |
| sim/modelo_referencia.py | Valores esperados y generador de src/rom_matriz.vhd. |
| sim/probar_matrices.py, sim/resultado_ghdl.txt | Prueba con las 8 matrices y sus resultados. |
| doc/figuras/*.png, doc/fuentes/*.py | Figuras de este documento y los scripts que las generan. |

## 2. Repaso del algoritmo de Bareiss

Bareiss es una eliminación gaussiana sin fracciones. En el paso k, cada elemento de la submatriz que está debajo y a la derecha del pivote se actualiza con la fórmula (1) del enunciado:

```
a(i,j) = ( a(k,k)·a(i,j) − a(i,k)·a(k,j) ) / p        para k < i ≤ n,  k < j ≤ n
p = pivote del paso anterior (p = 1 en el primer paso)
```

Después del paso k, el pivote actual a(k,k) pasa a ser p. Tras n − 1 pasos, el determinante queda en a(n,n), que es el último pivote. La división siempre es exacta: por la identidad de Sylvester, después del paso k cada a(i,j) es el determinante de una submatriz de (k+1)×(k+1) de A, que es un entero.

Durante el paso k solo cambian los elementos con i > k y j > k. La fila k y la columna k no se modifican, por eso el cálculo se hace sobre la misma RAM, sin una segunda copia de la matriz.

### Ejemplo 3×3

```
A = | 2 1 3 |      k = 1, p = 1:  a22 = (2·4 − 1·1)/1 = 7    a23 = (2·2 − 1·3)/1 = 1
    | 1 4 2 |                      a32 = (2·2 − 3·1)/1 = 1    a33 = (2·5 − 3·3)/1 = 1
    | 3 2 5 |      k = 2, p = 2:  a33 = (7·1 − 1·1)/2 = 3   →   det = 3
Banco de pivotes: pivote 1 = 2, pivote 2 = 7, pivote 3 = 3 = det
```

### Pivote cero

Si a(k,k) = 0, el ASM busca la primera fila i > k con a(i,k) ≠ 0 (estado s5), intercambia las filas k e i en las columnas k a n (s6 a s8) e invierte el bit `signo`. Al final, det = −a(n,n) si hubo un número impar de intercambios, y ese valor con su signo es el que se guarda como último pivote, para que el

pivote n siempre sea igual al determinante. Si no hay fila con a(i,k) ≠ 0, det = 0.

# 3. Caja negra

La caja negra es la del tablero de clase: entradas clk, rst, inicio y dir_p; salidas det, pivote y final. La ROM, la RAM, la división y el XS3 están adentro. Es la entidad `procesador`.

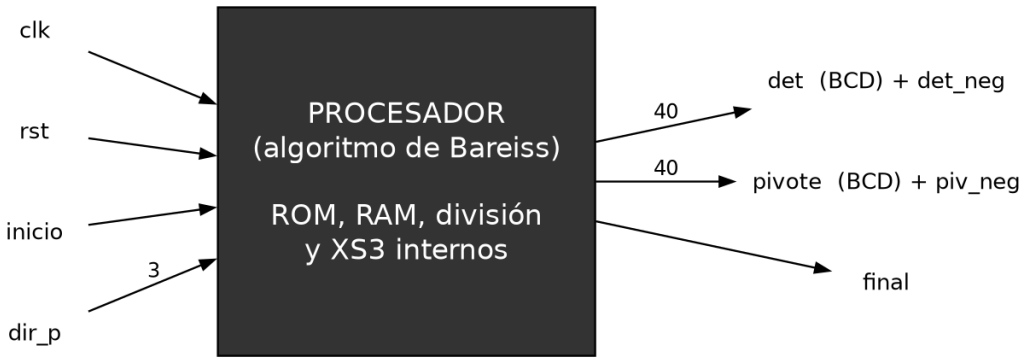



*Figura 1. Caja negra del procesador.*

| Puerto | Dir. | Bits | Función |
|---|---|---|---|
| clk | in | 1 | Reloj (100 MHz en la tarjeta) |
| rst | in | 1 | Reinicio asíncrono, activo en alto |
| inicio | in | 1 | Comienza el cálculo (pulso de un ciclo; se lee en s0 y s21) |
| dir_p | in | 3 | Dirección del banco de pivotes: 1 a n = pivote k (el n es el det); 0, 6 y 7 leen 0 |
| det, det_neg | out | 40, 1 | \|det\| en BCD (10 dígitos) y su signo |
| pivote, piv_neg | out | 40, 1 | \|pivote(dir_p)\| en BCD y su signo |
| final | out | 1 | '1' cuando det y pivote están listos (estado s21) |

## Caja negra de la tarjeta

La entidad `det_bareiss` conecta el procesador con la Nexys A7:

| Puerto | Pin | Función |
|---|---|---|
| clk | E3 | Reloj de 100 MHz |
| rst | BTNU (M18) | Reinicio de todo el diseño |
| inicio | BTNC (N17) | Comienza el cálculo (pasa por el antirrebote) |
| dir_p(2..0) | SW2..SW0 | 0: el display muestra det; 1 a 5: muestra el pivote k |
| final | LD0 | Fin del cálculo |
| desborde | LD1 | El valor mostrado tiene más de 7 dígitos |
| latido | LD15 | Onda de 1 s del divisor de frecuencia (indica que el diseño está corriendo) |
| an, seg, dp | AN7..AN0, CA..CG, DP | Display: AN7 = signo, AN6..AN0 = 7 dígitos |

# 4. Cajas grises

Antes del ASM se define el hardware: qué bloques hay, cómo se conectan y qué señales intercambian con la unidad de control. La caja gris del procesador muestra el ASM principal y los bloques del camino de datos con todas sus señales.

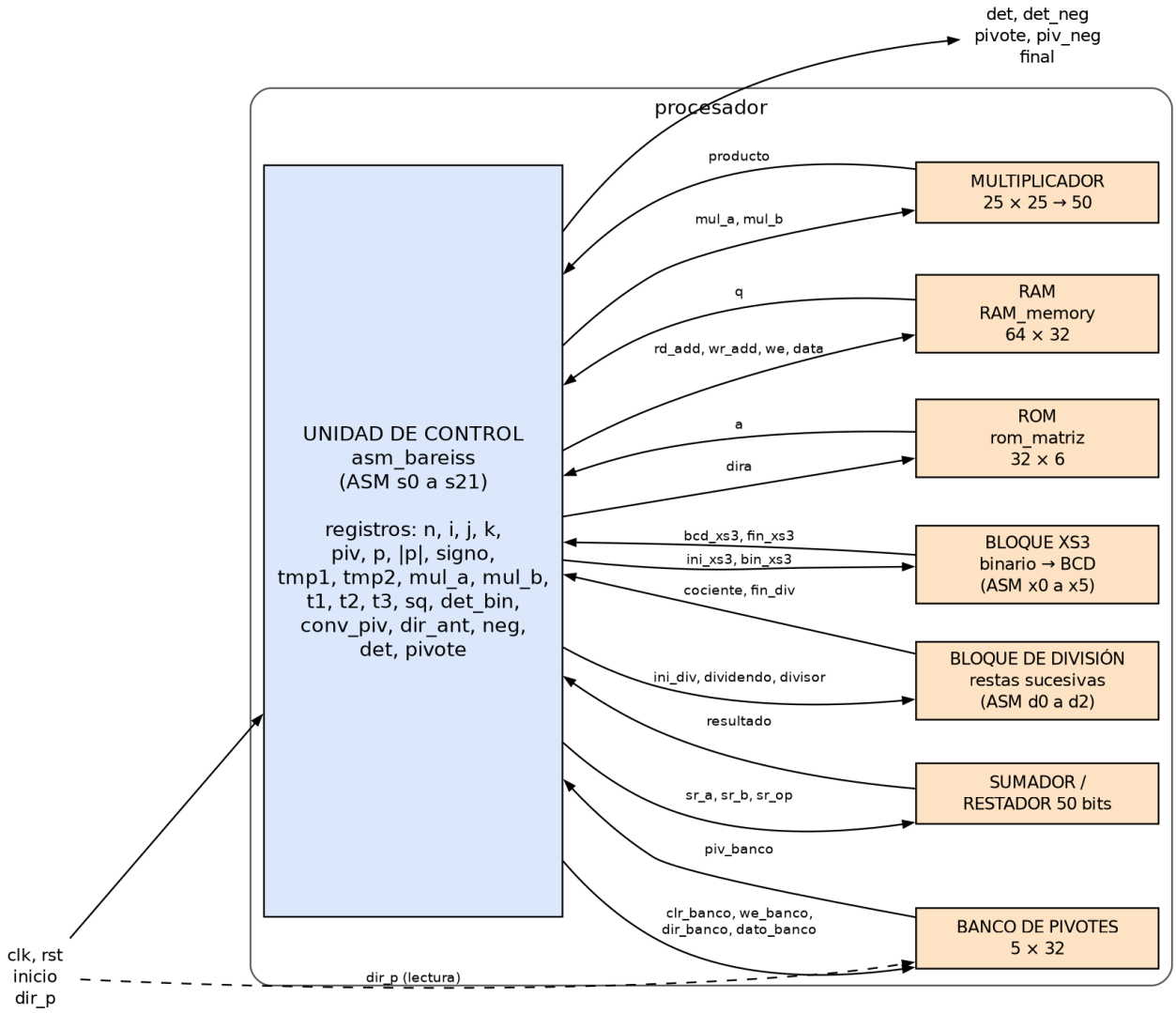



*Figura 2. Caja gris del procesador: unidad de control (ASM) y bloques del camino de datos.*

| Bloque | Entradas desde el ASM | Salidas hacia el ASM | Uso |
|---|---|---|---|
| ROM (rom_matriz) | dira (5) | a (6) | Leer n y los elementos (s1, s2) |
| RAM (RAM_memory) | rd_add (6), wr_add (6), we, data (32) | q (32) | Matriz de trabajo; dirección fila & columna |
| multiplicador | mul_a (25), mul_b (25) | producto (50) | a(k,k)·a(i,j) y a(i,k)·a(k,j) |
| sumador_restador | sr_a (50), sr_b (50), sr_op | resultado (50) | t1 − t2 y las negaciones 0 − x |
| bloque_division | ini_div, dividendo (50), divisor (32) | cociente (32), fin_div | \|t3\| / \|p\| por restas sucesivas |
| xs3 | ini_xs3, bin_xs3 (32) | bcd_xs3 (40), fin_xs3 | Convertir \|det\| y \|pivote\| a BCD |

| Bloque | Entradas desde el ASM | Salidas hacia el ASM | Uso |
|---|---|---|---|
| banco_pivotes | clr_banco, we_banco, dir_banco (3), dato_banco (32) | piv_banco (32) | Guardar los pivotes; se lee con dir_p |

## Cajas grises de los bloques con ASM propio

La división y el XS3 son pequeños procesadores: cada uno tiene una unidad de control y un camino de datos, y se comunican con el ASM principal con un protocolo inicio / final. El ASM principal activa `ini_div` o `ini_xs3` durante un ciclo y espera `fin_div` o `fin_xs3`.

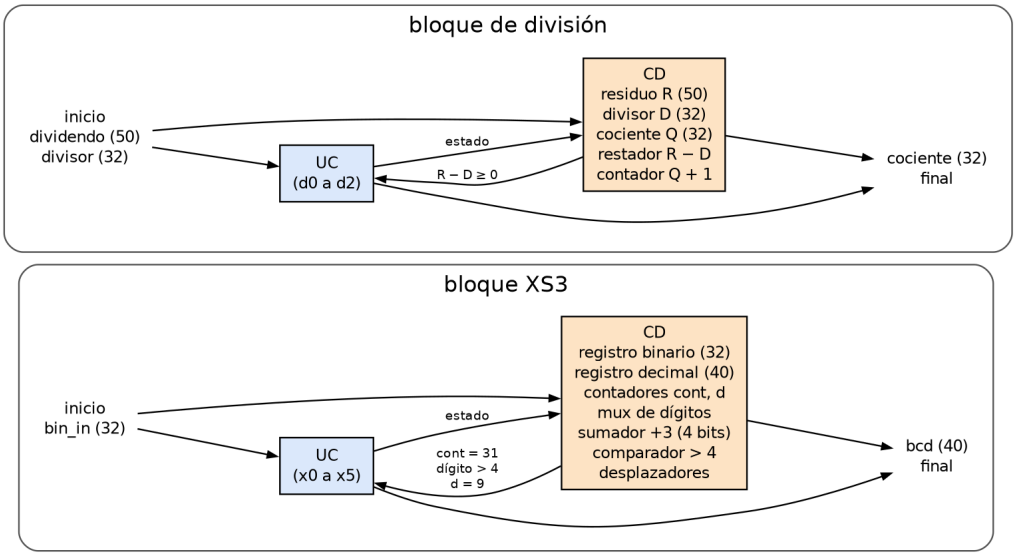



*Figura 3. Cajas grises del bloque XS3 y del bloque de división.*

## Tope de la tarjeta

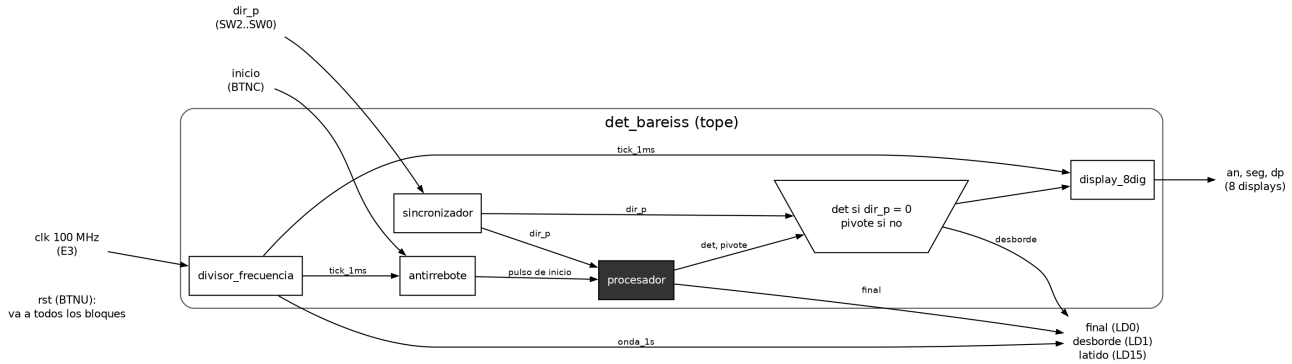



*Figura 4. Bloques del tope det_bareiss.*

# 5. Pseudoalgoritmo con el hardware disponible

Cada línea indica el estado del ASM principal que la ejecuta. Las operaciones del mismo estado ocurren en el mismo ciclo de reloj. «RAM(f,c)» es la palabra de la RAM en la dirección fila f, columna c.

```
s0 :  final = 0;  esperar inicio = 1
s1 :  n = ROM(00h)
s2 :  para j = 1..n, i = 1..n:  RAM(i,j) = ROM(dira);  dira = dira + 1     (por columnas)
```

```
s3 :  k = 1;  p = 1;  |p| = 1;  signo = 0;  conv_piv = 0;  banco = 0
mientras k <= n-1:
  s4 :  piv = RAM(k,k)
        si piv = 0:
  s5 :     buscar i = k+1..n con RAM(i,k) /= 0;  si no existe: det_bin = 0, ir a CONVERTIR
  s6-s8:   intercambiar filas k e i (columnas k..n);  signo = not signo;  volver a s4
  s9 :  banco(k) = piv
        para i = k+1..n, j = k+1..n:
  s10-s13:  t1 = piv · RAM(i,j);   t2 = RAM(i,k) · RAM(k,j)      (multiplicador)
  s14:      t3 = t1 - t2                                        (sumador/restador)
  s15-s16:  cociente = |t3| / |p|                               (bloque de división)
            RAM(i,j) = +cociente o -cociente  (signo(t3) xor signo(p))
  s17:  p = piv;  |p| = |piv|;  k = k + 1
s18:  det_bin = RAM(n,n) o -RAM(n,n) (según signo);  banco(n) = det_bin
CONVERTIR (dos veces: conv_piv = 0 para det, conv_piv = 1 para el pivote):
s19-s20:  BCD = XS3(|val|)  con val = det_bin o banco(dir_p)       (bloque XS3)
          si conv_piv = 0:  det = BCD;  conv_piv = 1;  repetir CONVERTIR
          si no:            pivote = BCD
s21:  final = 1;  si inicio = 1 ir a s1;  si cambia dir_p ir a CONVERTIR
```

Los «para» y «mientras» no son lazos del VHDL: cada uno es un lazo del ASM que vuelve a un estado mientras un contador no llegue a su límite.

# 6. Diagrama ASM principal

## Convenciones

| Elemento | Significado |
|---|---|
| Caja de estado (rectángulo azul) | Operaciones de todo el estado (salidas tipo Moore). |
| Caja de decisión (rombo) | Pregunta sobre una señal de estado. La salida 1 es la condición verdadera. |
| Caja de salida condicional (rectángulo redondeado naranja) | Operación que solo ocurre si se toma ese camino (tipo Mealy), como en el ejemplo «ASM Mealy» de clase. |
| Bloque ASM (recuadro punteado) | Caja de estado más sus decisiones y salidas condicionales: dura un ciclo de reloj. |
| Conector (círculo verde) | Une hojas del diagrama: A → s1, B → s4, C → s10, D → s19, E → s16. |
| x ← expresión | Carga de un registro; se hace efectiva en el flanco de subida al final del ciclo. |
| x = expresión | Señal combinacional hacia un bloque, por ejemplo ini_div = 1 o final = 1. |
| RAM(f,c) ← dato | Escritura en la RAM (we = 1) en la dirección fila f, columna c. |

Todas las decisiones de un bloque usan los valores que tienen los registros al comenzar el ciclo. Cada combinación de estado y decisiones lleva a un solo estado siguiente, y todo camino de decisiones termina en un estado.

## Vista general

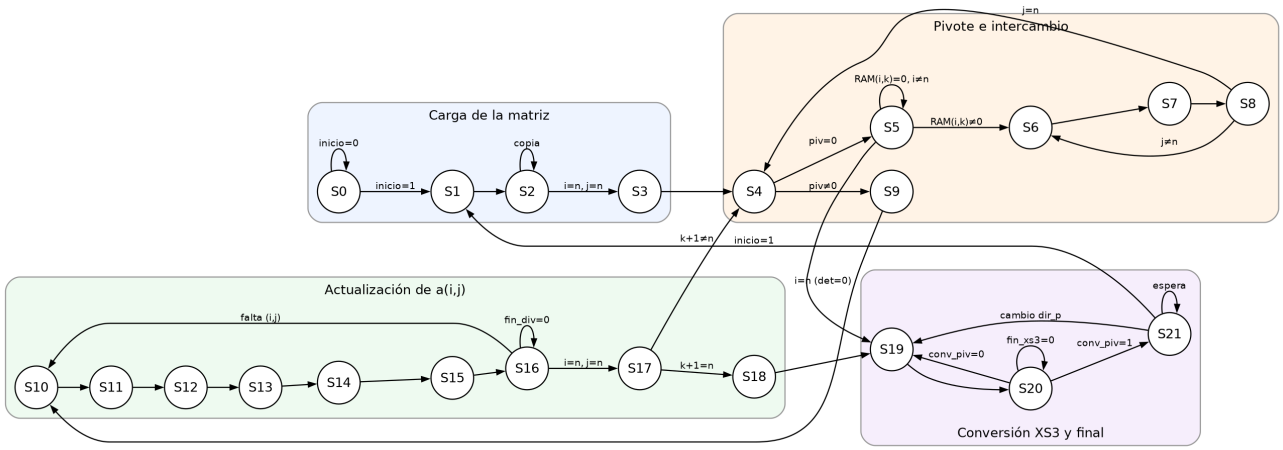



*Figura 5. Diagrama de estados del ASM principal (22 estados).*

## Hoja 1: espera y carga de la matriz (s0 a s3)

s2 recorre la ROM con el contador dira y los contadores i y j. La dirección de la RAM es fila & columna (3 + 3 bits), así que no hace falta multiplicar para ubicar a(i,j).

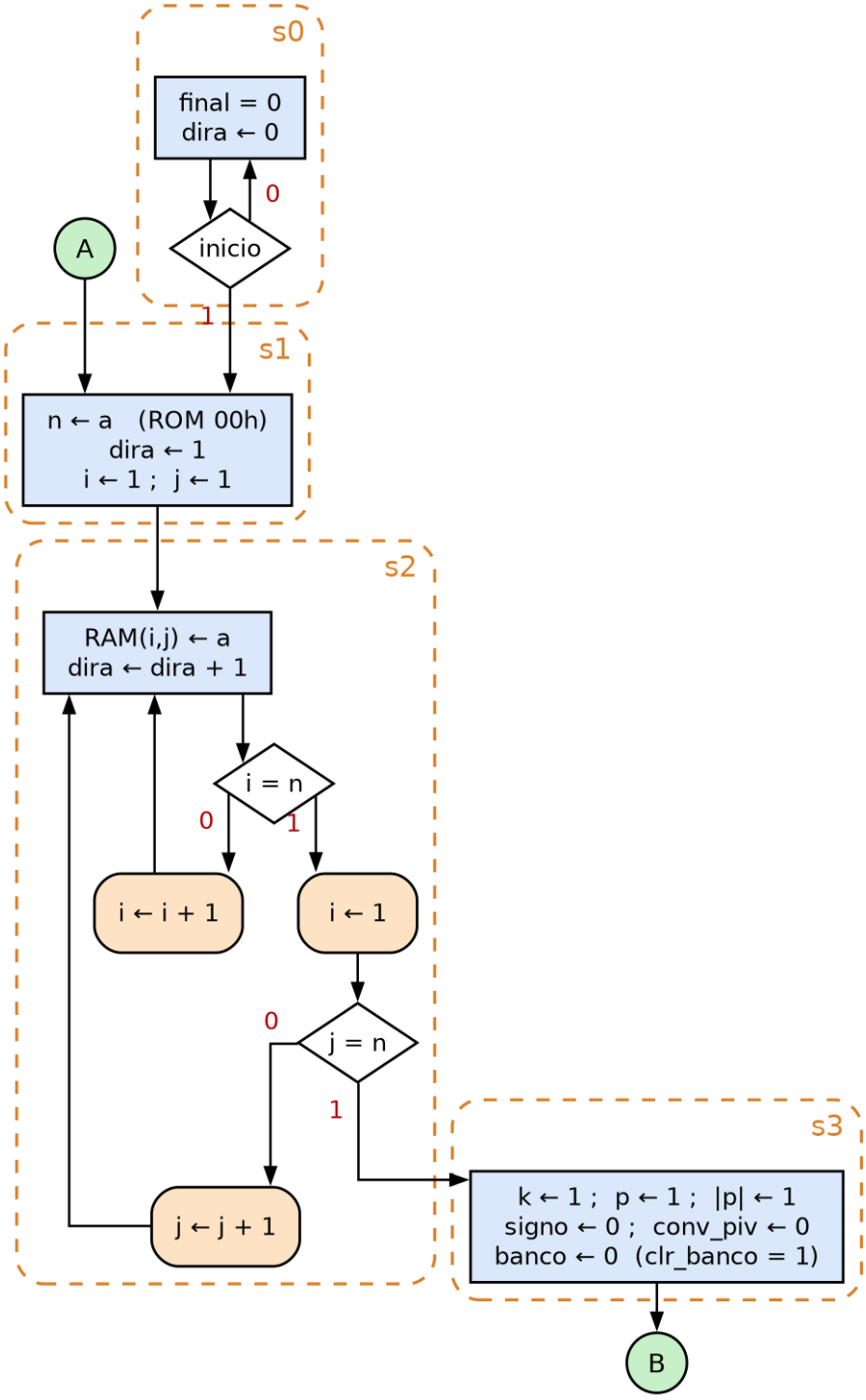



*Figura 6. Hoja 1: espera y carga de la matriz (s0 a s3).*

## Hoja 2: pivote, búsqueda de fila, intercambio y banco (s4 a s9)

La RAM tiene una dirección de lectura y otra de escritura. En s7 se lee RAM(i,j) y en el mismo flanco se escribe tmp1 en esa posición; por eso el intercambio usa tres estados por columna.

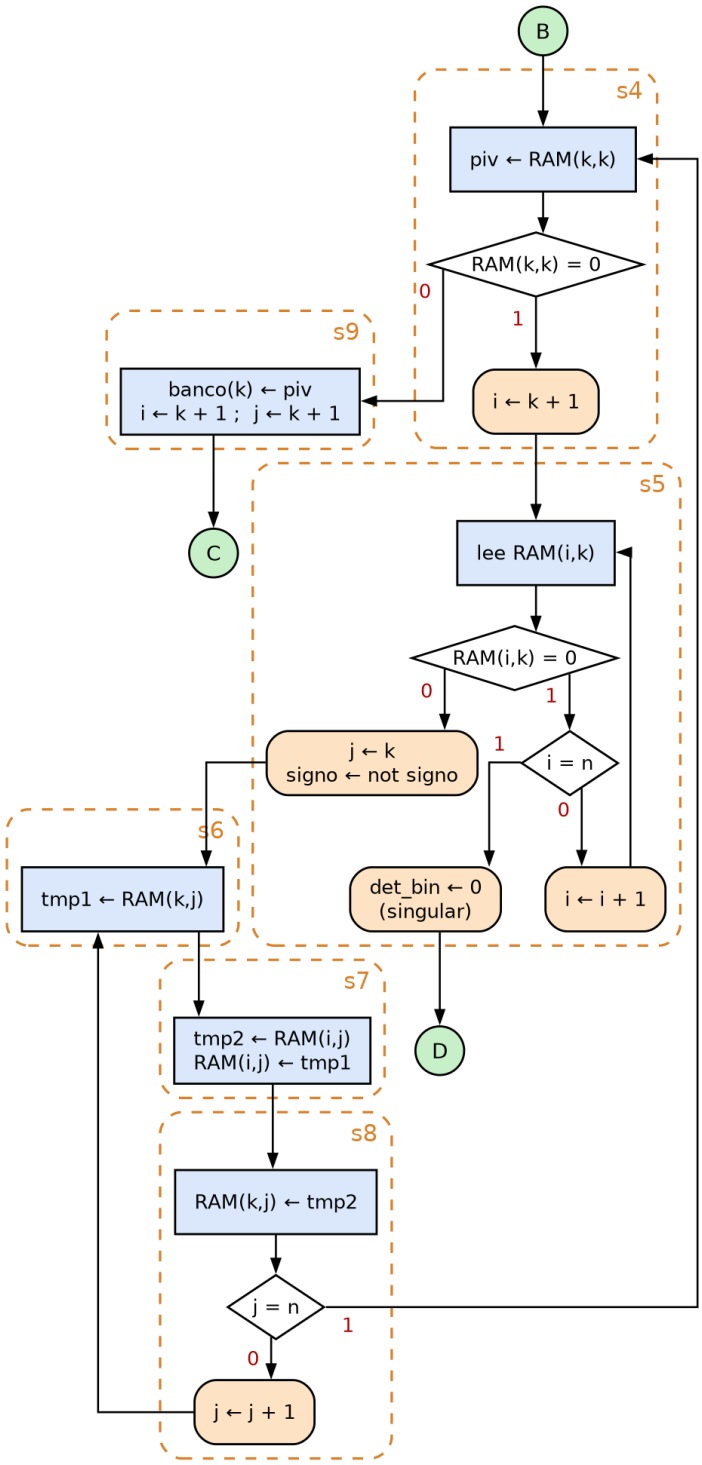



*Figura 7. Hoja 2: pivote, búsqueda de fila, intercambio y banco (s4 a s9).*

## Hoja 3: productos, resta e inicio de la división (s10 a s15)

Con un solo multiplicador, los dos productos se hacen en estados distintos. En s15 el sumador/restador calcula 0 − t3; el multiplexor de dividendo entrega |t3| y el bloque de división lo carga en el flanco.

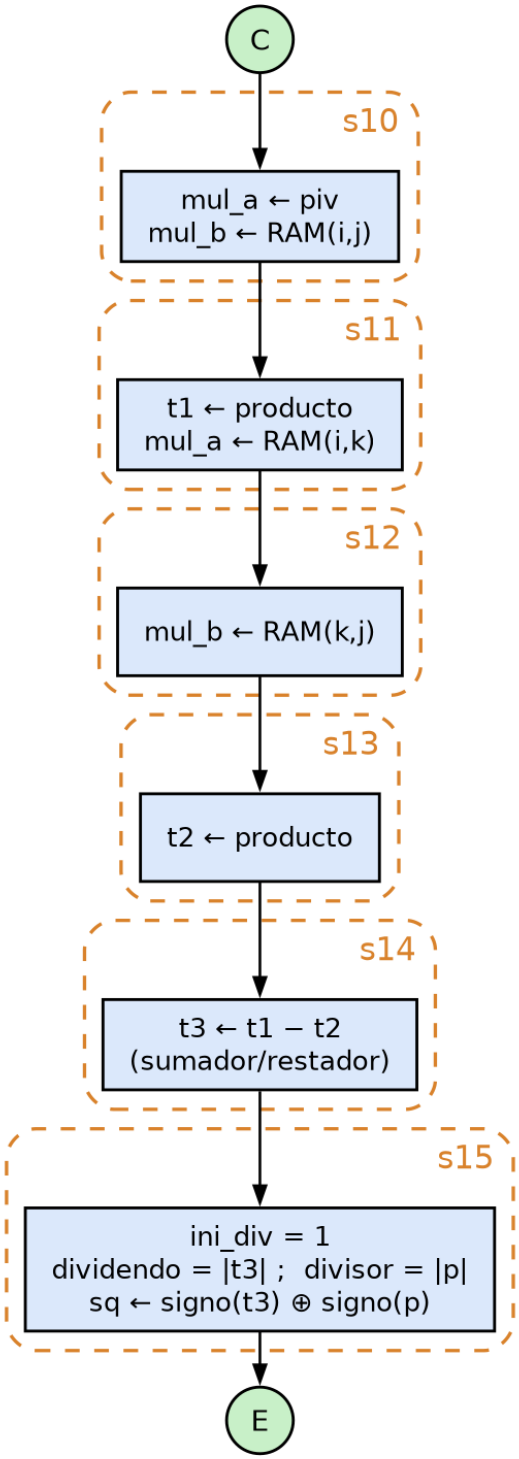



*Figura 8. Hoja 3: productos, resta e inicio de la división (s10 a s15).*

## Hoja 4: espera de la división, escritura y avance (s16 a s18)

El ASM espera fin_div. En ese ciclo el cociente es válido y se escribe en RAM(i,j) con el signo sq (el sumador/restador calcula 0 − cociente). s18 guarda el determinante como último pivote.

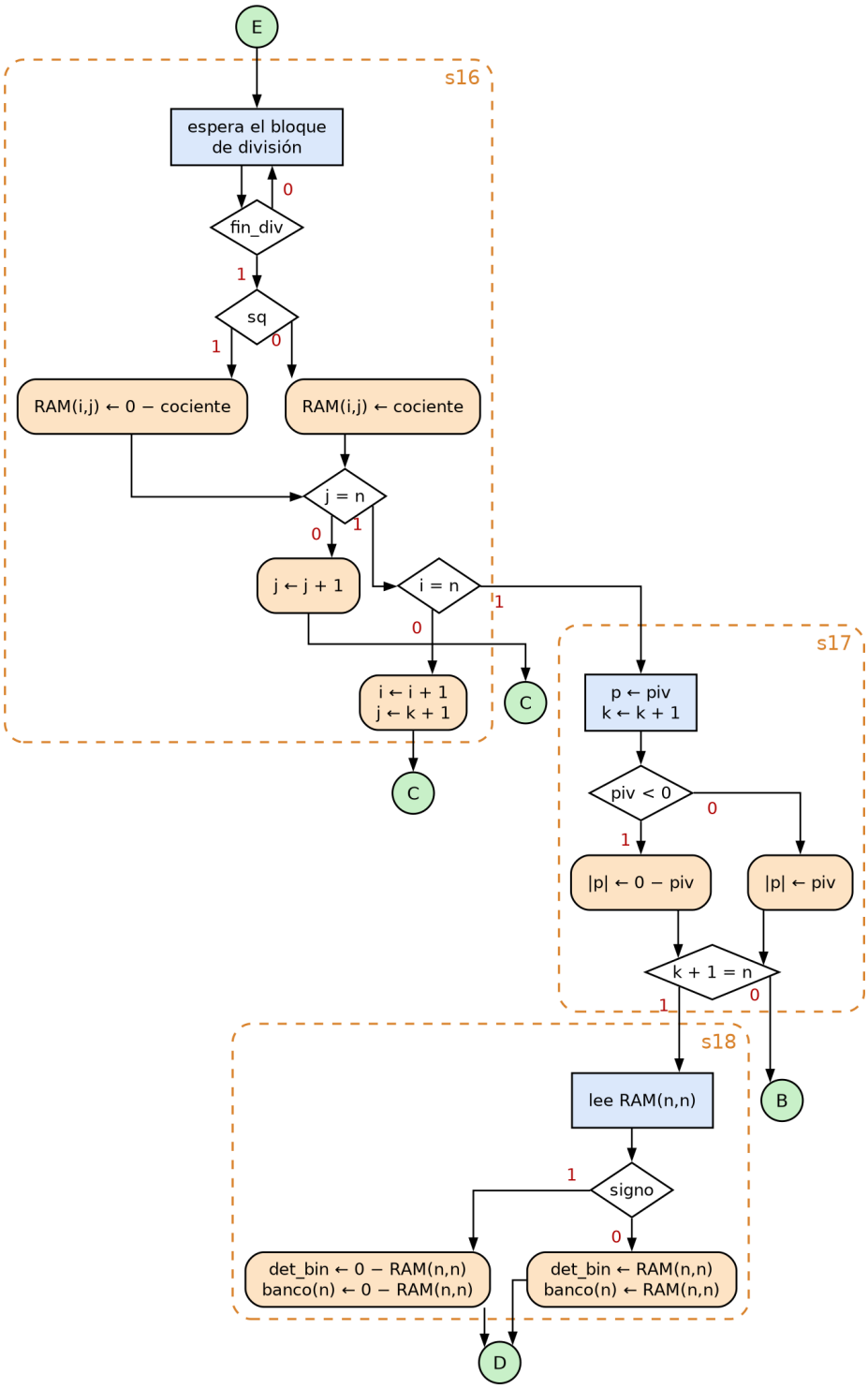



*Figura 9. Hoja 4: espera de la división, escritura y avance (s16 a s18).*

## Hoja 5: conversión XS3 y final (s19 a s21)

El bloque XS3 convierte primero |det| y luego |pivote(dir_p)|. En s21 el ASM espera con final = 1; si cambia dir_p vuelve a s19 y convierte el nuevo pivote (unos 4 µs).

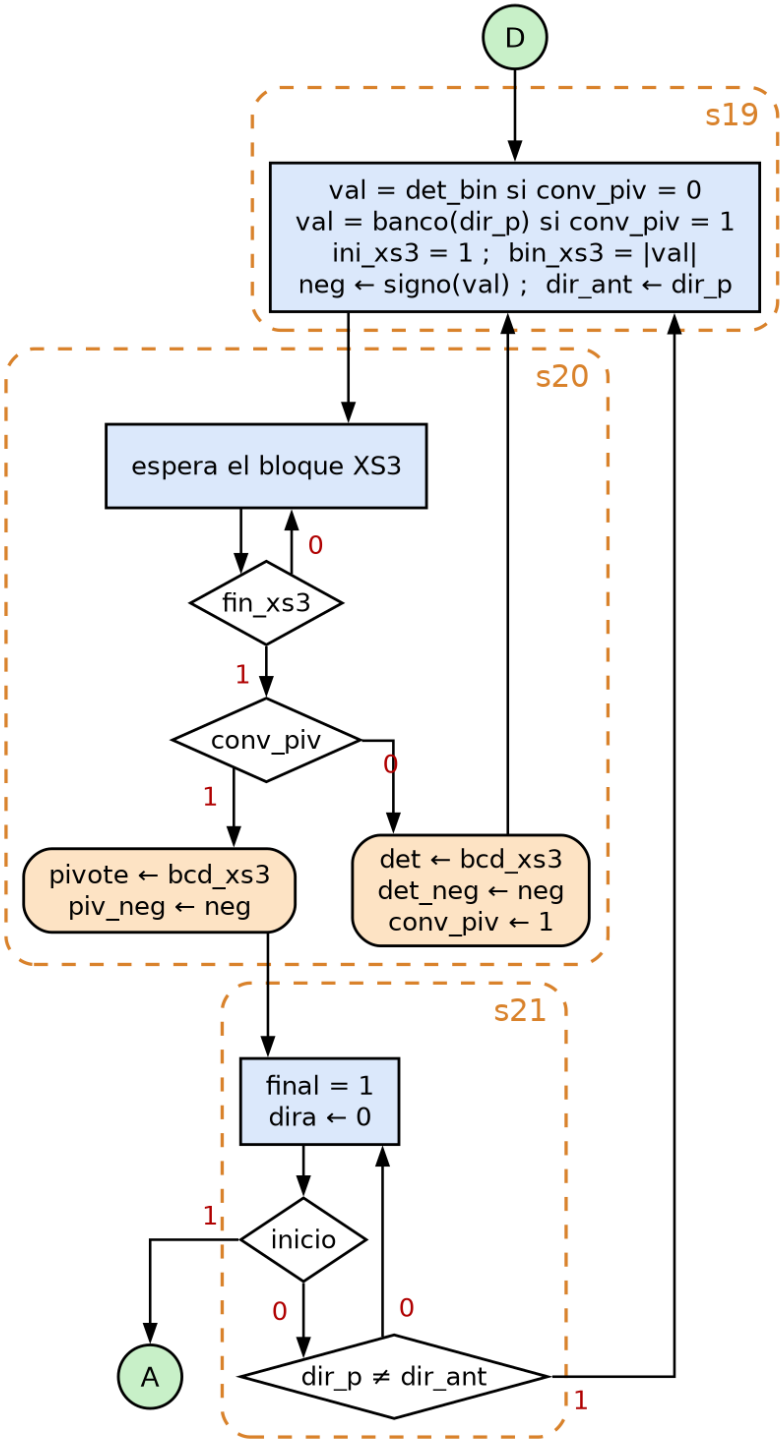



*Figura 10. Hoja 5: conversión XS3 y final (s19 a s21).*

## Tabla de estados

Resumen de cada bloque ASM, en el orden del archivo src/asm_bareiss.vhd.

| Estado | Operaciones | Siguiente estado |
|---|---|---|
| s0 | final = 0; dira ← 0 | inicio = 1: s1. Si no: s0. |
| s1 | n ← a (ROM 00h); dira ← 1; i ← 1; j ← 1 | s2 |
| s2 | RAM(i,j) ← a (con extensión de signo); dira ← dira + 1 | i ≠ n: i ← i + 1, s2. i = n y j ≠ n: i ← 1, j ← j + 1, s2. i = n y j = n: s3. |
| s3 | k ← 1; p ← 1; \|p\| ← 1; signo ← 0; conv_piv ← 0; banco ← 0 (clr_banco) | s4 |
| s4 | piv ← RAM(k,k) | RAM(k,k) = 0: i ← k + 1, s5. Si no: s9. |
| s5 | Lee RAM(i,k) | RAM(i,k) ≠ 0: j ← k, signo ← not signo, s6. RAM(i,k) = 0 e i = n: det_bin ← 0, s19. Si no: i ← i + 1, s5. |
| s6 | tmp1 ← RAM(k,j) | s7 |
| s7 | tmp2 ← RAM(i,j); RAM(i,j) ← tmp1 | s8 |
| s8 | RAM(k,j) ← tmp2 | j = n: s4. Si no: j ← j + 1, s6. |
| s9 | banco(k) ← piv; i ← k + 1; j ← k + 1 | s10 |
| s10 | mul_a ← piv; mul_b ← RAM(i,j) | s11 |
| s11 | t1 ← producto; mul_a ← RAM(i,k) | s12 |
| s12 | mul_b ← RAM(k,j) | s13 |
| s13 | t2 ← producto | s14 |
| s14 | t3 ← t1 − t2 | s15 |
| s15 | ini_div = 1; dividendo = \|t3\|; divisor = \|p\|; sq ← signo(t3) xor signo(p) | s16 |
| s16 | Espera el bloque de división | fin_div = 0: s16. fin_div = 1: RAM(i,j) ← ±cociente y además: j ≠ n: j ← j + 1, s10; j = n e i ≠ n: i ← i + 1, j ← k + 1, s10; i = n y j = n: s17. |
| s17 | p ← piv; \|p\| ← \|piv\|; k ← k + 1 | k + 1 = n: s18. Si no: s4. |
| s18 | det_bin ← ±RAM(n,n); banco(n) ← ±RAM(n,n) (signo negativo si signo = 1) | s19 |
| s19 | ini_xs3 = 1; bin_xs3 = \|val\| (val = det_bin si conv_piv = 0, banco(dir_p) si conv_piv = 1); neg ← signo(val); dir_ant ← dir_p | s20 |
| s20 | Espera el bloque XS3 | fin_xs3 = 0: s20. fin_xs3 = 1 y conv_piv = 0: det ← bcd_xs3, det_neg ← neg, conv_piv ← 1, s19. fin_xs3 = 1 y conv_piv = 1: pivote ← bcd_xs3, piv_neg ← neg, s21. |
| s21 | final = 1; dira ← 0 | inicio = 1: s1. dir_p ≠ dir_ant: s19. Si no: s21. |

# 7. ASM del bloque de división

El bloque `bloque_division` divide magnitudes por restas sucesivas. Recibe |t3| y |p|; el signo lo aplica el ASM principal al escribir en la RAM. Como la división de Bareiss es exacta, al terminar el residuo R es 0.

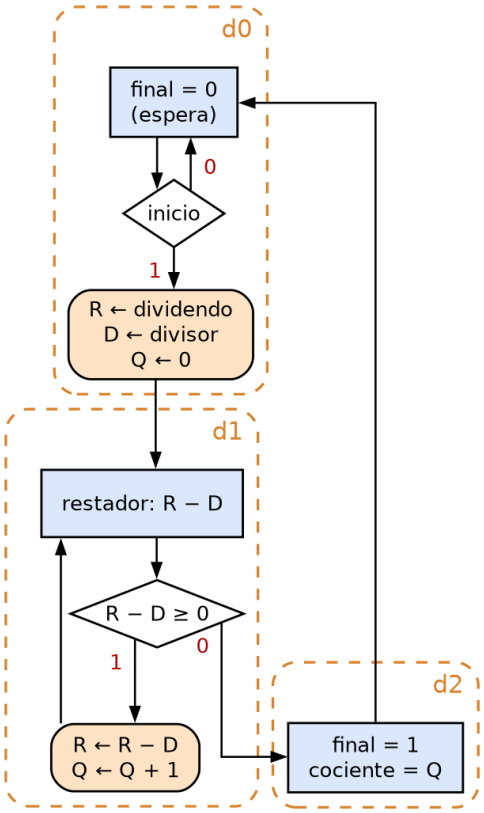



*Figura 11. ASM del bloque de división.*

| Estado | Operaciones | Siguiente estado |
|---|---|---|
| d0 | final = 0 | inicio = 1: R ← dividendo, D ← divisor, Q ← 0, d1. Si no: d0. |
| d1 | El restador calcula R − D | R − D ≥ 0: R ← R − D, Q ← Q + 1, d1. Si no: d2. |
| d2 | final = 1; cociente = Q | d0 |

Tarda |cociente| + 3 ciclos. El ASM principal está en s16 mientras tanto y escribe el cociente en el ciclo en que `final = 1`.

# 8. ASM del bloque XS3

El documento XS3 describe 5 pasos. En el bloque xs3 cada paso es un estado propio, de modo que el diagrama muestra la conversión completa. Su camino de datos es el que se describió en clase: registro binario, registro decimal, un sumador (+3), un comparador de magnitud (> 4) y desplazadores.

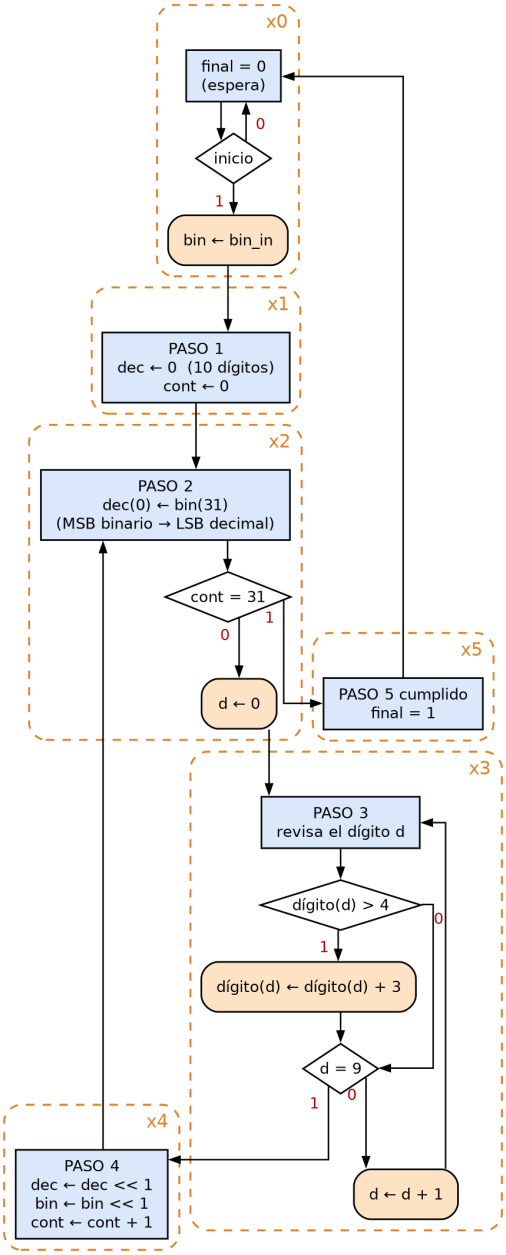



*Figura 12. ASM del bloque XS3.*

| Paso del documento XS3 | Estado | Operación |
|---|---|---|
| 1. Inicializar el registro decimal en ceros | x1 | dec ← 0 (40 bits); cont ← 0 |
| 2. Colocar el MSB del registro binario en el LSB del registro decimal | x2 | dec(0) ← bin(31) |
| 3. Revisar cada dígito: si es ≥ 5 (> 0100), sumarle 3 (0011) | x3 | Un dígito por ciclo (d = 0 a 9): el multiplexor entrega el dígito d; si es > 4 se reemplaza por dígito + 3. |
| 4. Correr a la izquierda y descartar el bit que sale; repetir desde el paso 2 | x4 | dec ← dec(38..0) & '0'; bin ← bin(30..0) & '0'; cont ← cont + 1; vuelve a x2 |
| 5. Realizar únicamente el paso 2 | x2 con cont = 31 | En la pasada 32 por x2 entra el último bit y el ASM va a x5 (final = 1). |

Por qué se suma 3: si un dígito vale 5 o más, al correrlo a la izquierda quedaría en 10 o más, que no es un dígito BCD válido. Sumar 3 antes del corrimiento equivale a sumar 6 después, que es la corrección BCD, y hace que el exceso pase como acarreo al dígito siguiente. El registro binario es de 32 bits, así que el decimal necesita 10 dígitos ($2^{32} - 1 = 4\,294\,967\,295$) y hay 32 pasadas por el paso 2.

## Traza del XS3 para 362

$362 = 1\,0110\,1010_2$ tiene 9 bits significativos. En las pasadas cont = 0 a 22 solo entran ceros, por eso la tabla empieza en cont = 23. «bin(31..23)» son los 9 bits de arriba del registro binario; el de la izquierda, bin(31), es el que copia el paso 2. El registro decimal se muestra con sus tres dígitos bajos.

| cont | Estado (paso) | dec: C D U | bin(31..23) | Comentario |
|---|---|---|---|---|
| 23 | x2 (2) | 0000 0000 0001 | 101101010 | |
| | x3 (3) | 0000 0000 0001 | 101101010 | ningún dígito > 4 |
| | x4 (4) | 0000 0000 0010 | 011010100 | corrimiento a la izquierda |
| 24 | x2 (2) | 0000 0000 0010 | 011010100 | |
| | x3 (3) | 0000 0000 0010 | 011010100 | ningún dígito > 4 |
| | x4 (4) | 0000 0000 0100 | 110101000 | corrimiento a la izquierda |
| 25 | x2 (2) | 0000 0000 0101 | 110101000 | |
| | x3 (3) | 0000 0000 1000 | 110101000 | BCD(0) = 0101 > 0100: +3 = 1000 |
| | x4 (4) | 0000 0001 0000 | 101010000 | corrimiento a la izquierda |
| 26 | x2 (2) | 0000 0001 0001 | 101010000 | |
| | x3 (3) | 0000 0001 0001 | 101010000 | ningún dígito > 4 |
| | x4 (4) | 0000 0010 0010 | 010100000 | corrimiento a la izquierda |
| 27 | x2 (2) | 0000 0010 0010 | 010100000 | |
| | x3 (3) | 0000 0010 0010 | 010100000 | ningún dígito > 4 |
| | x4 (4) | 0000 0100 0100 | 101000000 | corrimiento a la izquierda |
| 28 | x2 (2) | 0000 0100 0101 | 101000000 | |
| | x3 (3) | 0000 0100 1000 | 101000000 | BCD(0) = 0101 > 0100: +3 = 1000 |
| | x4 (4) | 0000 1001 0000 | 010000000 | corrimiento a la izquierda |
| 29 | x2 (2) | 0000 1001 0000 | 010000000 | |
| | x3 (3) | 0000 1100 0000 | 010000000 | BCD(1) = 1001 > 0100: +3 = 1100 |
| | x4 (4) | 0001 1000 0000 | 100000000 | corrimiento a la izquierda |
| 30 | x2 (2) | 0001 1000 0001 | 100000000 | |
| | x3 (3) | 0001 1011 0001 | 100000000 | BCD(1) = 1000 > 0100: +3 = 1011 |
| | x4 (4) | 0011 0110 0010 | 000000000 | corrimiento a la izquierda |
| 31 | x2 (2) | 0011 0110 0010 | 000000000 | último PASO 2 = PASO 5, va a x5 |

Resultado: dec = 0011 0110 0010 = 3 6 2.

# 9. Señales del ASM principal

## 9.1 Señales de control por estado (ASM → bloques)

Son asignaciones concurrentes de src/asm_bareiss.vhd que dependen del estado. Los estados que no aparecen no usan estos bloques.

| Estado | RAM (lectura / escritura) | Sumador/restador | Otros bloques |
|---|---|---|---|
| s2 | escribe (i,j) ← a | — | ROM: dira |
| s3 | — | — | banco: clr_banco = 1 |
| s4 | lee (k,k) | — | — |
| s5 | lee (i,k) | — | — |
| s6 | lee (k,j) | — | — |
| s7 | lee (i,j); escribe (i,j) ← tmp1 | — | — |
| s8 | escribe (k,j) ← tmp2 | — | — |
| s9 | — | — | banco: we_banco, dir_banco = k, dato = piv |
| s10 | lee (i,j) | — | multiplicador: carga mul_a, mul_b |
| s11 | lee (i,k) | — | multiplicador: t1 ← producto |
| s12 | lee (k,j) | — | — |
| s13 | — | — | multiplicador: t2 ← producto |
| s14 | — | t1 − t2 | — |
| s15 | — | 0 − t3 | división: ini_div, dividendo = \|t3\|, divisor = \|p\| |
| s16 | escribe (i,j) ← ±cociente cuando fin_div = 1 | 0 − cociente | — |
| s17 | — | 0 − piv | — |
| s18 | lee (n,n) | 0 − RAM(n,n) | banco: we_banco, dir_banco = n, dato = ±RAM(n,n) |
| s19 | — | 0 − val | XS3: ini_xs3, bin_xs3 = \|val\| |
| s21 | — | — | final = 1 |

En este algoritmo el sumador/restador siempre resta (`sr_op = '1'`): t1 − t2 y las negaciones 0 − x que dan los valores absolutos y los signos.

## 9.2 Señales de estado (bloques → ASM)

| Señal | Origen | Se pregunta en |
|---|---|---|
| inicio | Pulso del antirrebote | s0, s21 |
| RAM(k,k) = 0, RAM(i,k) = 0 | Comparador de q con 0 | s4, s5 |
| i = n, j = n | Comparadores de 3 bits | s2, s5, s8, s16 |
| k + 1 = n | Contador k y comparador | s17 |
| signo(t3) = t3(49) | Registro t3 | s15 (multiplexor de dividendo) |
| sq | Registro del signo del cociente | s16 |
| fin_div | Bloque de división | s16 |
| signo(piv) = piv(31) | Registro piv | s17 |
| signo | Registro de paridad de intercambios | s18 |
| signo(val) = val(31) | Multiplexor val | s19 |

| Señal | Origen | Se pregunta en |
|---|---|---|
| fin_xs3 | Bloque XS3 | s20 |
| conv_piv | Registro: 0 = convierte det, 1 = convierte pivote | s19, s20 |
| dir_p ≠ dir_ant | Comparador de 3 bits | s21 |

## 9.3 Registros del ASM principal y anchos

| Registro | Bits | Uso |
|---|---|---|
| st | 5 (22 estados) | Estado del ASM |
| dira_r | 5 | Dirección de la ROM |
| n, i, j, k | 3 c/u | Tamaño de la matriz y contadores de fila, columna y paso |
| piv, p, pabs | 32 c/u | Pivote actual, pivote anterior y \|p\| (divisor) |
| tmp1, tmp2 | 32 c/u | Intercambio de filas |
| mul_a, mul_b | 25 c/u | Entradas del multiplicador |
| t1, t2, t3 | 50 c/u | Productos y numerador de la fórmula (1) |
| sq, signo, conv_piv, neg | 1 c/u | Signo del cociente, paridad de intercambios, qué se convierte, signo convertido |
| det_bin | 32 | Determinante en binario |
| dir_ant | 3 | dir_p ya convertido |
| det, pivote | 40 c/u | Salidas en BCD |

Por la identidad de Sylvester, cada valor de la RAM es el determinante de una submatriz de A, y la desigualdad de Hadamard lo acota: $|\det| \le (32\cdot\sqrt{m})^m$ para una submatriz m×m.

- Operandos del multiplicador: submatrices de hasta 4×4, $|x| < 2^{24}$. Caben en 25 bits con signo.
- Determinante 5×5: $|\det| \le 1{,}88 \times 10^9 < 2^{31}$. RAM, det_bin, cociente y XS3 son de 32 bits.
- Numerador: $|t3| \le 3{,}2 \times 10^{14} < 2^{49}$. t1, t2, t3, el sumador/restador y el dividendo son de 50 bits.
- BCD: 32 bits binarios necesitan 10 dígitos. El display muestra 7 y LD1 avisa si hay más.

# 10. Interfaz con la tarjeta

## Divisor de frecuencia

El oscilador de la Nexys A7 es de 100 MHz (pin E3, sección 5 del manual de la tarjeta). El bloque `divisor_frecuencia` genera un pulso de un ciclo cada 1 ms (cuenta 100 000 ciclos) y una onda cuadrada de periodo 1 s (cambia cada 500 ms) que se ve en LD15. Los pulsos se usan como habilitación: todo el diseño trabaja con el mismo reloj, sin relojes derivados.

El procesador trabaja a 100 MHz y no usa el divisor. Con un reloj de 1 s, la matriz 3×3 de prueba (822 ciclos) tardaría unos 14 minutos y la 5×5 (4 583 ciclos) más de una hora.

## Antirrebote

Solo el botón de inicio lo necesita. El bloque `antirrebote` sincroniza el botón con dos flip-flops, lo muestrea cada 1 ms en un registro de 8 bits y cambia su nivel estable solo con 8 muestras iguales (8 ms sin rebote). Entrega un pulso de un ciclo en el flanco de subida del nivel estable, así una pulsación arranca un solo cálculo.

## Sincronizador y display

El bloque `sincronizador` pasa los switches dir_p por dos flip-flops. El bloque `display_8dig` cambia de dígito con cada pulso de 1 ms (barrido de 8 ms, 125 Hz): AN7 muestra el signo y AN6..AN0 los 7 dígitos. En el tope, un multiplexor elige det si dir_p = 0 y pivote si dir_p = 1 a 5, y LD1 se enciende si los dígitos 7 a 9 del valor mostrado no son cero.

# 11. Tiempo de ejecución

- Carga de la matriz (s0 a s3): $n^2 + 3$ ciclos.
- Por cada elemento actualizado: s10 a s15 (6 ciclos) más la división (|cociente| + 3 ciclos).
- Conversión XS3: unos 380 ciclos (3,8 µs). Al terminar el cálculo se hacen dos (det y pivote).

La división por restas sucesivas domina el tiempo cuando los valores son grandes. Las matrices de prueba terminan en menos de 50 µs, salvo la matriz 7 (valores cercanos a ±32), que tarda 19 millones de ciclos (0,19 s). Con los valores de la clase («del orden de cientos de miles») el tiempo es de microsegundos.

# 12. Implementación en ISE 14.7 y la Nexys A7

## 12.1 Proyecto

- File → New Project. Top-level source type: HDL. Family: Artix7 · Device: XC7A100T · Package: CSG324 · Speed: −1 · Synthesis: XST · Simulator: ISim · Preferred language: VHDL. ISE solo soporta la Nexys A7-100T.
- Add Source: todos los .vhd de src/ y src/nexys_a7.ucf (Implementation); sim/pkg_prueba.vhd, sim/tb_procesador.vhd y sim/tb_det_bareiss.vhd (Simulation). Si ISE no elige det_bareiss como tope, usen «Set as Top Module».
- Synthesize (View RTL Schematic muestra los bloques) → Implement Design → Generate Programming File. Revisen que se cumpla la restricción de 10 ns (TS_sys_clk).
- Programación: la Nexys A7 no funciona con Digilent Adept. Usen iMPACT de ISE o el Hardware Manager de Vivado (Lab Edition) con el archivo .bit.

## 12.2 Cambiar la matriz (día de la evaluación)

- Opción 1: editar src/rom_matriz.vhd. La línea de 00h es n; las siguientes son a11, a21, ..., an1, a12, ... en complemento a 2 de 6 bits.
- Opción 2: `python3 sim/modelo_referencia.py rom "2 1 3; 1 4 2; 3 2 5"` escribe el archivo e imprime el determinante y los pivotes esperados.
- Simular con sim/tb_procesador.vhd: lee la ROM, calcula los valores esperados y los compara, sin modificar el testbench.
- Sintetizar y volver a programar la FPGA.

## 12.3 Operación en la tarjeta

- Pulsar BTNU (rst). Poner SW2..SW0 en 000.
- Pulsar BTNC (inicio). Cuando termina, LD0 (final) se enciende y el display muestra el determinante.
- Poner SW2..SW0 en 1 a n para ver el pivote k; con k = n se ve otra vez el determinante.
- Si LD1 está encendido, el valor tiene más de 7 dígitos y se ven los 7 de abajo. LD15 parpadea cada segundo.

## 12.4 Matrices de prueba

| # | n | Matriz A | det | Pivotes (dir_p = 1..n) | Qué prueba |
|---|---|---|---|---|---|
| 0 | 2 | 7 -3<br>-3 5 | 26 | 7, 26 | Caso básico 2×2 |
| 1 | 3 | 2 1 3<br>1 4 2<br>3 2 5 | 3 | 2, 7, 3 | Ejemplo del documento |
| 2 | 4 | 0 2 1 3<br>2 1 0 1<br>1 0 3 2<br>3 1 2 4 | -22 | 2, 4, 13, -22 | a11 = 0: intercambio en k = 1 |
| 3 | 5 | 4 -2 1 3 0<br>-2 6 2 -1 5<br>1 2 -7 4 2<br>3 -1 4 5 -3<br>0 5 2 -3 8 | 362 | 4, 20, -170, -639, 362 | 5×5 general (ROM por defecto) |
| 4 | 3 | -2 4 1<br>4 -8 3<br>1 3 5 | 50 | -2, -10, 50 | Pivote negativo; intercambio en k = 2 |
| 5 | 4 | 1 2 3 0<br>2 4 1 5<br>3 1 -2 4<br>0 5 4 -3 | 150 | 1, -5, 25, 150 | Pivote cero en k = 2 (p ≠ 1) |
| 6 | 5 | 0 0 0 0 0<br>0 3 1 2 4<br>0 1 5 -2 1<br>0 2 -2 6 3<br>0 4 1 3 7 | 0 | 0, 0, 0, 0, 0 | Columna 1 en cero: singular |
| 7 | 5 | 30 27 24 -29 -31<br>27 -21 28 29 -24<br>24 28 2 -28 -28<br>-29 29 -28 -27 28<br>-31 -24 -28 28 -30 | -17 121 828 | 30, -1359, 22146, 277564, -17121828 | Valores extremos; desborde del display |

src/rom_matriz.vhd trae la matriz 3. Para cargar otra de la lista: `python3 sim/modelo_referencia.py rom <número>`.

### 12.5 Pines (src/nexys_a7.ucf)

| Señal | Pines |
|---|---|
| clk | E3 (100 MHz) |
| rst / inicio | M18 (BTNU) / N17 (BTNC) |
| dir_p(0..2) | J15 (SW0), L16 (SW1), M13 (SW2) |
| final / desborde / latido | H17 (LD0) / K15 (LD1) / V11 (LD15) |
| seg(0..6) = CA..CG, dp | T10, R10, K16, K13, P15, T11, L18; DP = H15 |
| an(0..7) | J17, J18, T9, J14, P14, T14, K2, U13 |

## 13. Simulación

- **sim/tb_procesador.vhd** (con sim/pkg_prueba.vhd) lee la matriz de la ROM, calcula el determinante y los pivotes esperados con un modelo de Bareiss en VHDL (con números reales para no desbordar los enteros de 32 bits), pulsa inicio, compara det, recorre dir_p = 1 a n comparando cada pivote, comprueba que el pivote n sea el det y repite todo con un segundo inicio.
- **sim/probar_matrices.py** escribe cada una de las 8 matrices de prueba en una ROM temporal y corre ese testbench con GHDL.
- **sim/tb_det_bareiss.vhd** prueba la tarjeta completa con tiempos reales: pulsa BTNC con rebotes, comprueba que hay un solo cálculo y lee los 8 dígitos decodificando an y seg con dir_p = 0, 2 y n.

| # | n | det esperado | det del procesador | Ciclos hasta final | Resultado |
|---|---|---|---|---|---|
| 0 | 2 | ' 0000000026' | ' 0000000026' | 795 | OK |
| 1 | 3 | ' 0000000003' | ' 0000000003' | 822 | OK |
| 2 | 4 | '-0000000022' | '-0000000022' | 1 003 | OK |
| 3 | 5 | ' 0000000362' | ' 0000000362' | 4 583 | OK |
| 4 | 3 | ' 0000000050' | ' 0000000050' | 898 | OK |
| 5 | 4 | ' 0000000150' | ' 0000000150' | 1 184 | OK |
| 6 | 5 | ' 0000000000' | ' 0000000000' | 783 | OK |
| 7 | 5 | '-0017121828' | '-0017121828' | 19 013 805 | OK |

Todos los pivotes también coinciden con el modelo, y en la prueba de la tarjeta el display mostró « 0000362» con un solo cálculo después de la pulsación con rebotes (ver sim/resultado_ghdl.txt). Para comprobar que las pruebas detectan errores se probaron tres fallas en una copia del diseño: XS3 sumando 2 en lugar de 3, antirrebote sin el muestreo de 1 ms (el cálculo arrancó 13 veces) y último pivote sin el signo de los intercambios (matriz 2). Las tres se detectaron.

## 14. Limitaciones y verificación pendiente

- **No se ejecutó ISE.** El diseño se verificó con GHDL 4.1 (simulación con `-fsynopsys -fexplicit` y síntesis genérica). Hay que sintetizar en ISE 14.7 y revisar el reporte de tiempos a 100 MHz. El camino más largo pasa por el sumador/restador de 50 bits hacia el bloque de división (s15); si no se cumplen los 10 ns, generen un reloj de 50 MHz con el Clocking Wizard y pongan PERIOD = 20 ns.
- **Salidas en BCD.** det y pivote salen del procesador ya convertidos por el XS3. Si el profesor las quiere en binario, los valores existen dentro del ASM (det_bin y el banco).

- **Multiplicador con signo.** Usa el paquete `std_logic_signed` sobre std_logic_vector (no el tipo signed). Si no se acepta, hay que multiplicar magnitudes y aplicar el signo con el sumador/restador.
- **RAM.** Es el modelo del curso con 64 palabras en lugar de 32 y contenido inicial en cero.
- **Display de 7 dígitos.** Valores hasta ±9 999 999; para valores mayores se enciende LD1.
- **Valor de n.** El diseño supone $2 \leq n \leq 5$ en la ROM y no lo valida.

# Anexo A. Código VHDL

## src/det_bareiss.vhd

```
1  ----------------------------------------------------------------------------------
2  -- Universidad Distrital Francisco Jose de Caldas
3  -- Analisis y Diseno de Microprocesadores - Taller 5
4  --
5  -- TOPE PARA LA NEXYS A7-100T (descripcion estructural)
6  --
7  --   divisor_frecuencia -> tick de 1 ms y onda de 1 s
8  --   antirrebote        -> un pulso de inicio por cada pulsacion de BTNC
9  --   sincronizador      -> switches dir_p (SW2..SW0)
10 --   procesador         -> ASM de Bareiss con sus bloques (ROM, RAM, division, XS3...)
11 --   display_8dig       -> muestra det (dir_p = 0) o el pivote dir_p (1..5)
12 --
13 -- Herramienta: Xilinx ISE 14.7. Reloj de la tarjeta: 100 MHz (pin E3).
14 ----------------------------------------------------------------------------------
15 library IEEE;
16 use IEEE.STD_LOGIC_1164.ALL;
17
18 entity det_bareiss is
19     Port ( clk      : in  STD_LOGIC;                      -- 100 MHz
20            rst      : in  STD_LOGIC;                      -- BTNU
21            inicio   : in  STD_LOGIC;                      -- BTNC
22            dir_p    : in  STD_LOGIC_VECTOR (2 downto 0);  -- SW2..SW0
23            final    : out STD_LOGIC;                      -- LD0
24            desborde : out STD_LOGIC;                      -- LD1: mas de 7 digitos
25            latido   : out STD_LOGIC;                      -- LD15: onda de 1 s
26            an       : out STD_LOGIC_VECTOR (7 downto 0);
27            seg      : out STD_LOGIC_VECTOR (6 downto 0);
28            dp       : out STD_LOGIC);
29 end det_bareiss;
30
31 architecture Estructural of det_bareiss is
32
33 component divisor_frecuencia
34     Port ( clk, rst : in  STD_LOGIC;
35            tick_1ms : out STD_LOGIC;
36            onda_1s  : out STD_LOGIC);
37 end component;
38
39 component antirrebote
40     Port ( clk, rst : in  STD_LOGIC;
41            tick     : in  STD_LOGIC;
42            boton    : in  STD_LOGIC;
43            pulso    : out STD_LOGIC);
44 end component;
45
46 component sincronizador
47     Port ( clk, rst : in  STD_LOGIC;
48            entrada  : in  STD_LOGIC_VECTOR (2 downto 0);
49            salida   : out STD_LOGIC_VECTOR (2 downto 0));
50 end component;
51
52 component procesador
53     Port ( clk, rst : in  STD_LOGIC;
54            inicio   : in  STD_LOGIC;
55            dir_p    : in  STD_LOGIC_VECTOR (2 downto 0);
56            det      : out STD_LOGIC_VECTOR (39 downto 0);
57            det_neg  : out STD_LOGIC;
58            pivote   : out STD_LOGIC_VECTOR (39 downto 0);
59            piv_neg  : out STD_LOGIC;
60            final    : out STD_LOGIC);
61 end component;
62
63 component display_8dig
64     Port ( clk, rst : in  STD_LOGIC;
65            tick     : in  STD_LOGIC;
66            bcd      : in  STD_LOGIC_VECTOR (27 downto 0);
67            neg      : in  STD_LOGIC;
68            an       : out STD_LOGIC_VECTOR (7 downto 0);
69            seg      : out STD_LOGIC_VECTOR (6 downto 0);
70            dp       : out STD_LOGIC);
71 end component;
72
73 constant CERO12 : STD_LOGIC_VECTOR (11 downto 0) := x"000";
74
75 signal tick_1ms, pulso_inicio : STD_LOGIC;
76 signal dir_p_s     : STD_LOGIC_VECTOR (2 downto 0);
77 signal det, pivote, mostrar : STD_LOGIC_VECTOR (39 downto 0);
78 signal det_neg, piv_neg, mostrar_neg : STD_LOGIC;
79
80 begin
81
```

```
U_FREC : divisor_frecuencia port map (clk => clk, rst => rst, tick_1ms => tick_1ms, onda_1s => latido);

U_ANTI : antirrebote port map (clk => clk, rst => rst, tick => tick_1ms, boton => inicio, pulso => pulso_inicio);

U_SINC : sincronizador port map (clk => clk, rst => rst, entrada => dir_p, salida => dir_p_s);

U_PROC : procesador port map (
    clk => clk, rst => rst, inicio => pulso_inicio, dir_p => dir_p_s,
    det => det, det_neg => det_neg, pivote => pivote, piv_neg => piv_neg, final => final);

-- Que se muestra: dir_p = 0 -> determinante ; dir_p = 1..5 -> pivote dir_p
mostrar     <= det     when dir_p_s = "000" else pivote;
mostrar_neg <= det_neg when dir_p_s = "000" else piv_neg;
desborde    <= '0' when mostrar(39 downto 28) = CERO12 else '1';

U_DISP : display_8dig port map (
    clk => clk, rst => rst, tick => tick_1ms, bcd => mostrar(27 downto 0), neg => mostrar_neg,
    an => an, seg => seg, dp => dp);

end Estructural;
```

## src/procesador.vhd

```
--------------------------------------------------------------------------------
-- PROCESADOR: unidad de control (ASM) + camino de datos (descripcion estructural)
--
-- Caja negra:
--   entradas : clk, rst, inicio, dir_p (direccion del pivote, 1..n)
--   salidas  : det (BCD) + det_neg, pivote (BCD) + piv_neg, final
--
-- Bloques internos (cada uno en su archivo):
--   asm_bareiss       ASM principal (unidad de control y registros del algoritmo)
--   rom_matriz        ROM con la matriz (direccion dira, dato a)
--   RAM_memory        RAM de trabajo (modelo del curso)
--   multiplicador     unico multiplicador
--   sumador_restador  unico sumador/restador
--   bloque_division   division por restas sucesivas (ASM propio)
--   xs3               conversion binario -> BCD (ASM propio)
--   banco_pivotes     banco de registros de los pivotes
--------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity procesador is
    Port ( clk, rst : in  STD_LOGIC;
           inicio   : in  STD_LOGIC;
           dir_p    : in  STD_LOGIC_VECTOR (2 downto 0);
           det      : out STD_LOGIC_VECTOR (39 downto 0);
           det_neg  : out STD_LOGIC;
           pivote   : out STD_LOGIC_VECTOR (39 downto 0);
           piv_neg  : out STD_LOGIC;
           final    : out STD_LOGIC);
end procesador;

architecture Estructural of procesador is

component asm_bareiss
    Port ( clk, rst   : in  STD_LOGIC;
           inicio     : in  STD_LOGIC;
           dir_p      : in  STD_LOGIC_VECTOR (2 downto 0);
           dira       : out STD_LOGIC_VECTOR (4 downto 0);
           a          : in  STD_LOGIC_VECTOR (5 downto 0);
           rd_add     : out STD_LOGIC_VECTOR (5 downto 0);
           wr_add     : out STD_LOGIC_VECTOR (5 downto 0);
           we         : out STD_LOGIC;
           data       : out STD_LOGIC_VECTOR (31 downto 0);
           q          : in  STD_LOGIC_VECTOR (31 downto 0);
           mul_a      : out STD_LOGIC_VECTOR (24 downto 0);
           mul_b      : out STD_LOGIC_VECTOR (24 downto 0);
           producto   : in  STD_LOGIC_VECTOR (49 downto 0);
           sr_a       : out STD_LOGIC_VECTOR (49 downto 0);
           sr_b       : out STD_LOGIC_VECTOR (49 downto 0);
           sr_op      : out STD_LOGIC;
           resultado  : in  STD_LOGIC_VECTOR (49 downto 0);
           ini_div    : out STD_LOGIC;
           dividendo  : out STD_LOGIC_VECTOR (49 downto 0);
           divisor    : out STD_LOGIC_VECTOR (31 downto 0);
           cociente   : in  STD_LOGIC_VECTOR (31 downto 0);
           fin_div    : in  STD_LOGIC;
           clr_banco  : out STD_LOGIC;
           we_banco   : out STD_LOGIC;
           dir_banco  : out STD_LOGIC_VECTOR (2 downto 0);
           dato_banco : out STD_LOGIC_VECTOR (31 downto 0);
           piv_banco  : in  STD_LOGIC_VECTOR (31 downto 0);
           ini_xs3    : out STD_LOGIC;
           bin_xs3    : out STD_LOGIC_VECTOR (31 downto 0);
           bcd_xs3    : in  STD_LOGIC_VECTOR (39 downto 0);
```

```
            fin_xs3    : in  STD_LOGIC;
            det        : out STD_LOGIC_VECTOR (39 downto 0);
            det_neg    : out STD_LOGIC;
            pivote     : out STD_LOGIC_VECTOR (39 downto 0);
            piv_neg    : out STD_LOGIC;
            final      : out STD_LOGIC);
end component;

component rom_matriz
    Port ( dira : in  STD_LOGIC_VECTOR (4 downto 0);
           a    : out STD_LOGIC_VECTOR (5 downto 0));
end component;

component RAM_memory
    port ( c      : in  std_logic;
           data   : in  std_logic_vector (31 downto 0);
           wr_add : in  std_logic_vector (5 downto 0);
           rd_add : in  std_logic_vector (5 downto 0);
           we     : in  std_logic;
           q      : out std_logic_vector (31 downto 0));
end component;

component multiplicador
    Port ( a : in  STD_LOGIC_VECTOR (24 downto 0);
           b : in  STD_LOGIC_VECTOR (24 downto 0);
           p : out STD_LOGIC_VECTOR (49 downto 0));
end component;

component sumador_restador
    Port ( a  : in  STD_LOGIC_VECTOR (49 downto 0);
           b  : in  STD_LOGIC_VECTOR (49 downto 0);
           op : in  STD_LOGIC;
           s  : out STD_LOGIC_VECTOR (49 downto 0));
end component;

component bloque_division
    Port ( clk, rst  : in  STD_LOGIC;
           inicio    : in  STD_LOGIC;
           dividendo : in  STD_LOGIC_VECTOR (49 downto 0);
           divisor   : in  STD_LOGIC_VECTOR (31 downto 0);
           cociente  : out STD_LOGIC_VECTOR (31 downto 0);
           final     : out STD_LOGIC);
end component;

component xs3
    Port ( clk, rst : in  STD_LOGIC;
           inicio   : in  STD_LOGIC;
           bin_in   : in  STD_LOGIC_VECTOR (31 downto 0);
           bcd      : out STD_LOGIC_VECTOR (39 downto 0);
           final    : out STD_LOGIC);
end component;

component banco_pivotes
    Port ( clk, rst : in  STD_LOGIC;
           clr      : in  STD_LOGIC;
           we       : in  STD_LOGIC;
           dir_w    : in  STD_LOGIC_VECTOR (2 downto 0);
           dato     : in  STD_LOGIC_VECTOR (31 downto 0);
           dir_r    : in  STD_LOGIC_VECTOR (2 downto 0);
           q        : out STD_LOGIC_VECTOR (31 downto 0));
end component;

-- ROM
signal dira : STD_LOGIC_VECTOR (4 downto 0);
signal a    : STD_LOGIC_VECTOR (5 downto 0);
-- RAM
signal rd_add, wr_add : STD_LOGIC_VECTOR (5 downto 0);
signal we             : STD_LOGIC;
signal data, q        : STD_LOGIC_VECTOR (31 downto 0);
-- multiplicador
signal mul_a, mul_b : STD_LOGIC_VECTOR (24 downto 0);
signal producto     : STD_LOGIC_VECTOR (49 downto 0);
-- sumador/restador
signal sr_a, sr_b, resultado : STD_LOGIC_VECTOR (49 downto 0);
signal sr_op                 : STD_LOGIC;
-- division
signal ini_div, fin_div : STD_LOGIC;
signal dividendo        : STD_LOGIC_VECTOR (49 downto 0);
signal divisor, cociente : STD_LOGIC_VECTOR (31 downto 0);
-- banco de pivotes
signal clr_banco, we_banco : STD_LOGIC;
signal dir_banco           : STD_LOGIC_VECTOR (2 downto 0);
signal dato_banco, piv_banco : STD_LOGIC_VECTOR (31 downto 0);
-- XS3
signal ini_xs3, fin_xs3 : STD_LOGIC;
signal bin_xs3          : STD_LOGIC_VECTOR (31 downto 0);
signal bcd_xs3          : STD_LOGIC_VECTOR (39 downto 0);

```

```
begin

U_ASM : asm_bareiss port map (
    clk => clk, rst => rst, inicio => inicio, dir_p => dir_p,
    dira => dira, a => a,
    rd_add => rd_add, wr_add => wr_add, we => we, data => data, q => q,
    mul_a => mul_a, mul_b => mul_b, producto => producto,
    sr_a => sr_a, sr_b => sr_b, sr_op => sr_op, resultado => resultado,
    ini_div => ini_div, dividendo => dividendo, divisor => divisor,
    cociente => cociente, fin_div => fin_div,
    clr_banco => clr_banco, we_banco => we_banco, dir_banco => dir_banco,
    dato_banco => dato_banco, piv_banco => piv_banco,
    ini_xs3 => ini_xs3, bin_xs3 => bin_xs3, bcd_xs3 => bcd_xs3, fin_xs3 => fin_xs3,
    det => det, det_neg => det_neg, pivote => pivote, piv_neg => piv_neg,
    final => final);

U_ROM : rom_matriz port map (dira => dira, a => a);

U_RAM : RAM_memory port map (
    c => clk, data => data, wr_add => wr_add, rd_add => rd_add, we => we, q => q);

U_MUL : multiplicador port map (a => mul_a, b => mul_b, p => producto);

U_SR : sumador_restador port map (a => sr_a, b => sr_b, op => sr_op, s => resultado);

U_DIV : bloque_division port map (
    clk => clk, rst => rst, inicio => ini_div, dividendo => dividendo,
    divisor => divisor, cociente => cociente, final => fin_div);

U_XS3 : xs3 port map (
    clk => clk, rst => rst, inicio => ini_xs3, bin_in => bin_xs3,
    bcd => bcd_xs3, final => fin_xs3);

U_BANCO : banco_pivotes port map (
    clk => clk, rst => rst, clr => clr_banco, we => we_banco, dir_w => dir_banco,
    dato => dato_banco, dir_r => dir_p, q => piv_banco);

end Estructural;
```

## src/asm_bareiss.vhd

```
----------------------------------------------------------------------------------
-- ASM PRINCIPAL DEL PROCESADOR (algoritmo de Bareiss)
--
-- Descripcion de alto nivel del diagrama ASM (s0 a s21): un "when" por estado,
-- las cajas de decision son "if" y las salidas condicionales son las
-- asignaciones dentro del "if". Los bloques del camino de datos (ROM, RAM,
-- multiplicador, sumador/restador, division, XS3 y banco de pivotes) estan
-- fuera de este archivo: el ASM les entrega direcciones, datos y senales de
-- inicio, y lee sus resultados por los puertos.
--
-- Solo std_logic y std_logic_vector. Los estados son constantes.
----------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.std_logic_unsigned.all;

entity asm_bareiss is
    Port ( clk, rst   : in  STD_LOGIC;
           inicio     : in  STD_LOGIC;
           dir_p      : in  STD_LOGIC_VECTOR (2 downto 0);
           -- ROM
           dira       : out STD_LOGIC_VECTOR (4 downto 0);
           a          : in  STD_LOGIC_VECTOR (5 downto 0);
           -- RAM
           rd_add     : out STD_LOGIC_VECTOR (5 downto 0);
           wr_add     : out STD_LOGIC_VECTOR (5 downto 0);
           we         : out STD_LOGIC;
           data       : out STD_LOGIC_VECTOR (31 downto 0);
           q          : in  STD_LOGIC_VECTOR (31 downto 0);
           -- multiplicador
           mul_a      : out STD_LOGIC_VECTOR (24 downto 0);
           mul_b      : out STD_LOGIC_VECTOR (24 downto 0);
           producto   : in  STD_LOGIC_VECTOR (49 downto 0);
           -- sumador/restador
           sr_a       : out STD_LOGIC_VECTOR (49 downto 0);
           sr_b       : out STD_LOGIC_VECTOR (49 downto 0);
           sr_op      : out STD_LOGIC;
           resultado  : in  STD_LOGIC_VECTOR (49 downto 0);
           -- bloque de division
           ini_div    : out STD_LOGIC;
           dividendo  : out STD_LOGIC_VECTOR (49 downto 0);
           divisor    : out STD_LOGIC_VECTOR (31 downto 0);
           cociente   : in  STD_LOGIC_VECTOR (31 downto 0);
           fin_div    : in  STD_LOGIC;
           -- banco de pivotes
           clr_banco  : out STD_LOGIC;
```

```
           we_banco   : out STD_LOGIC;
           dir_banco  : out STD_LOGIC_VECTOR (2 downto 0);
           dato_banco : out STD_LOGIC_VECTOR (31 downto 0);
           piv_banco  : in  STD_LOGIC_VECTOR (31 downto 0);   -- banco(dir_p)
           -- bloque XS3
           ini_xs3    : out STD_LOGIC;
           bin_xs3    : out STD_LOGIC_VECTOR (31 downto 0);
           bcd_xs3    : in  STD_LOGIC_VECTOR (39 downto 0);
           fin_xs3    : in  STD_LOGIC;
           -- salidas del procesador
           det        : out STD_LOGIC_VECTOR (39 downto 0);   -- |det| en BCD
           det_neg    : out STD_LOGIC;
           pivote     : out STD_LOGIC_VECTOR (39 downto 0);   -- |pivote(dir_p)| en BCD
           piv_neg    : out STD_LOGIC;
           final      : out STD_LOGIC);
end asm_bareiss;

architecture Behavioral of asm_bareiss is

-------------------------------------------------------------------------------
-- Estados
-------------------------------------------------------------------------------
-- s0        espera de inicio
-- s1..s3    lectura de n, copia ROM -> RAM, inicializacion
-- s4..s9    pivote: lectura, busqueda de fila, intercambio, banco(k)
-- s10..s15  productos, resta e inicio de la division
-- s16       espera de la division y escritura de a(i,j)
-- s17       p = pivote actual, k = k + 1
-- s18       det = +/- a(n,n) ; banco(n) = det
-- s19..s20  conversion XS3 de det y del pivote elegido
-- s21       final = 1
constant s0  : STD_LOGIC_VECTOR (4 downto 0) := "00000";
constant s1  : STD_LOGIC_VECTOR (4 downto 0) := "00001";
constant s2  : STD_LOGIC_VECTOR (4 downto 0) := "00010";
constant s3  : STD_LOGIC_VECTOR (4 downto 0) := "00011";
constant s4  : STD_LOGIC_VECTOR (4 downto 0) := "00100";
constant s5  : STD_LOGIC_VECTOR (4 downto 0) := "00101";
constant s6  : STD_LOGIC_VECTOR (4 downto 0) := "00110";
constant s7  : STD_LOGIC_VECTOR (4 downto 0) := "00111";
constant s8  : STD_LOGIC_VECTOR (4 downto 0) := "01000";
constant s9  : STD_LOGIC_VECTOR (4 downto 0) := "01001";
constant s10 : STD_LOGIC_VECTOR (4 downto 0) := "01010";
constant s11 : STD_LOGIC_VECTOR (4 downto 0) := "01011";
constant s12 : STD_LOGIC_VECTOR (4 downto 0) := "01100";
constant s13 : STD_LOGIC_VECTOR (4 downto 0) := "01101";
constant s14 : STD_LOGIC_VECTOR (4 downto 0) := "01110";
constant s15 : STD_LOGIC_VECTOR (4 downto 0) := "01111";
constant s16 : STD_LOGIC_VECTOR (4 downto 0) := "10000";
constant s17 : STD_LOGIC_VECTOR (4 downto 0) := "10001";
constant s18 : STD_LOGIC_VECTOR (4 downto 0) := "10010";
constant s19 : STD_LOGIC_VECTOR (4 downto 0) := "10011";
constant s20 : STD_LOGIC_VECTOR (4 downto 0) := "10100";
constant s21 : STD_LOGIC_VECTOR (4 downto 0) := "10101";

signal st : STD_LOGIC_VECTOR (4 downto 0);

-------------------------------------------------------------------------------
-- Constantes
-------------------------------------------------------------------------------
constant UNO3   : STD_LOGIC_VECTOR (2 downto 0)  := "001";
constant CERO18 : STD_LOGIC_VECTOR (17 downto 0) := "000000000000000000";
constant CERO32 : STD_LOGIC_VECTOR (31 downto 0) := x"00000000";
constant UNO32  : STD_LOGIC_VECTOR (31 downto 0) := x"00000001";
constant CERO40 : STD_LOGIC_VECTOR (39 downto 0) := x"0000000000";
constant CERO50 : STD_LOGIC_VECTOR (49 downto 0) := (others => '0');

-------------------------------------------------------------------------------
-- Registros del ASM
-------------------------------------------------------------------------------
signal dira_r     : STD_LOGIC_VECTOR (4 downto 0);   -- direccion de la ROM
signal n, i, j, k : STD_LOGIC_VECTOR (2 downto 0);   -- tamano y contadores
signal piv        : STD_LOGIC_VECTOR (31 downto 0);  -- pivote actual a(k,k)
signal p          : STD_LOGIC_VECTOR (31 downto 0);  -- pivote anterior
signal pabs       : STD_LOGIC_VECTOR (31 downto 0);  -- |p| (divisor)
signal signo      : STD_LOGIC;                       -- paridad de intercambios
signal tmp1, tmp2 : STD_LOGIC_VECTOR (31 downto 0);  -- intercambio de filas
signal mul_a_r, mul_b_r : STD_LOGIC_VECTOR (24 downto 0);
signal t1, t2, t3 : STD_LOGIC_VECTOR (49 downto 0);  -- productos y numerador
signal sq         : STD_LOGIC;                       -- signo del cociente
signal det_bin    : STD_LOGIC_VECTOR (31 downto 0);  -- determinante en binario
signal conv_piv   : STD_LOGIC;                       -- '0' convierte det, '1' el pivote
signal dir_ant    : STD_LOGIC_VECTOR (2 downto 0);   -- dir_p ya convertido
signal neg        : STD_LOGIC;                       -- signo del valor convertido
signal det_bcd, piv_bcd     : STD_LOGIC_VECTOR (39 downto 0);
signal det_neg_r, piv_neg_r : STD_LOGIC;

-------------------------------------------------------------------------------
-- Senales combinacionales
```

```
-------------------------------------------------------------------------------
signal dira_mas1 : STD_LOGIC_VECTOR (4 downto 0);    -- contadores
signal i_mas1, j_mas1, k_mas1 : STD_LOGIC_VECTOR (2 downto 0);
signal ext_a   : STD_LOGIC_VECTOR (25 downto 0);     -- extensiones de signo
signal a32     : STD_LOGIC_VECTOR (31 downto 0);
signal sx_piv, sx_q, sx_val : STD_LOGIC_VECTOR (17 downto 0);
signal rd_fila, rd_col, wr_fila : STD_LOGIC_VECTOR (2 downto 0);
signal val     : STD_LOGIC_VECTOR (31 downto 0);     -- valor que se convierte
signal det_nn  : STD_LOGIC_VECTOR (31 downto 0);     -- +/- a(n,n)

begin

-------------------------------------------------------------------------------
-- Contadores y extensiones de signo
-------------------------------------------------------------------------------
dira_mas1 <= dira_r + '1';
i_mas1    <= i + '1';
j_mas1    <= j + '1';
k_mas1    <= k + '1';

ext_a  <= (others => a(5));
a32    <= ext_a & a;
sx_piv <= (others => piv(31));
sx_q   <= (others => q(31));
sx_val <= (others => val(31));

-------------------------------------------------------------------------------
-- Senales hacia la ROM y la RAM
-------------------------------------------------------------------------------
dira <= dira_r;

rd_fila <= k when (st = s4 or st = s6 or st = s12) else      -- M(k,k), M(k,j)
           n when st = s18 else                              -- M(n,n)
           i;
rd_col  <= k when (st = s4 or st = s5 or st = s11) else      -- M(k,k), M(i,k)
           n when st = s18 else
           j;
rd_add  <= rd_fila & rd_col;

wr_fila <= k when st = s8 else i;                           -- M(k,j) en s8
wr_add  <= wr_fila & j;

we   <= '1' when (st = s2 or st = s7 or st = s8 or (st = s16 and fin_div = '1')) else '0';

data <= a32                    when st = s2 else             -- copia de la ROM
        tmp1                   when st = s7 else             -- intercambio
        tmp2                   when st = s8 else             -- intercambio
        resultado(31 downto 0) when sq = '1' else            -- s16: -cociente
        cociente;                                            -- s16: +cociente

-------------------------------------------------------------------------------
-- Multiplicador
-------------------------------------------------------------------------------
mul_a <= mul_a_r;
mul_b <= mul_b_r;

-------------------------------------------------------------------------------
-- Sumador/restador (unico): operandos segun el estado
-------------------------------------------------------------------------------
sr_a <= t1 when st = s14 else CERO50;                        -- t1 - t2 ; 0 - x
sr_b <= t2              when st = s14 else
        t3              when st = s15 else                   -- -t3
        CERO18 & cociente when st = s16 else                 -- -cociente
        sx_piv & piv    when st = s17 else                   -- -piv
        sx_q & q        when st = s18 else                   -- -a(n,n)
        sx_val & val    when st = s19 else                   -- -valor
        CERO50;
sr_op <= '1';                     -- en este algoritmo todas las operaciones son restas

-------------------------------------------------------------------------------
-- Bloque de division: |t3| / |p|
-------------------------------------------------------------------------------
ini_div   <= '1' when st = s15 else '0';
dividendo <= resultado when t3(49) = '1' else t3;            -- |t3|
divisor   <= pabs;

-------------------------------------------------------------------------------
-- Banco de pivotes
-------------------------------------------------------------------------------
det_nn     <= resultado(31 downto 0) when signo = '1' else q; -- s18
clr_banco  <= '1' when st = s3 else '0';
we_banco   <= '1' when (st = s9 or st = s18) else '0';
dir_banco  <= n when st = s18 else k;
dato_banco <= det_nn when st = s18 else piv;

-------------------------------------------------------------------------------
-- Bloque XS3: convierte |det| (conv_piv = 0) o |pivote(dir_p)| (conv_piv = 1)
-------------------------------------------------------------------------------
```

```
val     <= det_bin when conv_piv = '0' else piv_banco;
ini_xs3 <= '1' when st = s19 else '0';
bin_xs3 <= resultado(31 downto 0) when val(31) = '1' else val;  -- |val|

-------------------------------------------------------------------------------
-- Salidas del procesador
-------------------------------------------------------------------------------
det     <= det_bcd;
det_neg <= det_neg_r;
pivote  <= piv_bcd;
piv_neg <= piv_neg_r;
final   <= '1' when st = s21 else '0';

-------------------------------------------------------------------------------
-- DIAGRAMA ASM
-------------------------------------------------------------------------------
process (clk, rst)
begin

if rst = '1' then
    st       <= s0;
    dira_r   <= "00000";
    n <= "000";  i <= "000";  j <= "000";  k <= "000";
    piv      <= CERO32;  p <= CERO32;  pabs <= CERO32;  signo <= '0';
    tmp1     <= CERO32;  tmp2 <= CERO32;
    mul_a_r  <= (others => '0');  mul_b_r <= (others => '0');
    t1       <= CERO50;  t2 <= CERO50;  t3 <= CERO50;  sq <= '0';
    det_bin  <= CERO32;
    conv_piv <= '0';     dir_ant <= "000";  neg <= '0';
    det_bcd  <= CERO40;  det_neg_r <= '0';
    piv_bcd  <= CERO40;  piv_neg_r <= '0';

elsif clk'event and clk = '1' then

    case st is

        ---------------------------------------------------------------
        -- Espera y carga de la matriz
        ---------------------------------------------------------------
        when s0 =>                              -- final = 0 ; dira = 0
            dira_r <= "00000";
            if inicio = '0' then
                st <= s0;
            else
                st <= s1;
            end if;

        when s1 =>                              -- n = ROM(00h)
            n      <= a(2 downto 0);
            dira_r <= "00001";
            i      <= UNO3;
            j      <= UNO3;
            st     <= s2;

        when s2 =>                              -- RAM(i,j) = ROM(dira), por columnas
            dira_r <= dira_mas1;
            if i = n then
                i <= UNO3;
                if j = n then
                    st <= s3;
                else
                    j  <= j_mas1;
                    st <= s2;
                end if;
            else
                i  <= i_mas1;
                st <= s2;
            end if;

        when s3 =>                              -- inicializacion ; banco = 0
            k        <= UNO3;
            p        <= UNO32;                  -- pivote inicial a(0,0) = 1
            pabs     <= UNO32;
            signo    <= '0';
            conv_piv <= '0';
            st       <= s4;

        ---------------------------------------------------------------
        -- Pivote del paso k
        ---------------------------------------------------------------
        when s4 =>                              -- piv = M(k,k)
            piv <= q;
            if q = CERO32 then
                i  <= k_mas1;
                st <= s5;
            else
                st <= s9;
            end if;
```

```
311
312         when s5 =>                              -- busca fila i > k con M(i,k) /= 0
313             if q /= CERO32 then
314                 j     <= k;
315                 signo <= not signo;             -- el intercambio cambia el signo
316                 st    <= s6;
317             elsif i = n then
318                 det_bin <= CERO32;              -- no hay fila: matriz singular
319                 st      <= s19;
320             else
321                 i  <= i_mas1;
322                 st <= s5;
323             end if;
324
325         when s6 =>                              -- tmp1 = M(k,j)
326             tmp1 <= q;
327             st   <= s7;
328
329         when s7 =>                              -- tmp2 = M(i,j) ; RAM(i,j) = tmp1
330             tmp2 <= q;
331             st   <= s8;
332
333         when s8 =>                              -- RAM(k,j) = tmp2
334             if j = n then
335                 st <= s4;                       -- vuelve a leer el pivote
336             else
337                 j  <= j_mas1;
338                 st <= s6;
339             end if;
340
341         when s9 =>                              -- banco(k) = piv ; i = j = k + 1
342             i  <= k_mas1;
343             j  <= k_mas1;
344             st <= s10;
345
346         ------------------------------------------------------------------
347         -- a(i,j) = ( a(k,k)*a(i,j) - a(i,k)*a(k,j) ) / p
348         ------------------------------------------------------------------
349         when s10 =>                             -- mul_a = a(k,k) ; mul_b = M(i,j)
350             mul_a_r <= piv(24 downto 0);
351             mul_b_r <= q(24 downto 0);
352             st      <= s11;
353
354         when s11 =>                             -- t1 = producto ; mul_a = M(i,k)
355             t1      <= producto;
356             mul_a_r <= q(24 downto 0);
357             st      <= s12;
358
359         when s12 =>                             -- mul_b = M(k,j)
360             mul_b_r <= q(24 downto 0);
361             st      <= s13;
362
363         when s13 =>                             -- t2 = producto
364             t2 <= producto;
365             st <= s14;
366
367         when s14 =>                             -- t3 = t1 - t2
368             t3 <= resultado;
369             st <= s15;
370
371         when s15 =>                             -- inicio division |t3| / |p|
372             sq <= t3(49) xor p(31);             -- signo del cociente
373             st <= s16;
374
375         when s16 =>                             -- espera la division
376             if fin_div = '1' then               -- RAM(i,j) = +/- cociente
377                 if j = n then
378                     if i = n then
379                         st <= s17;
380                     else
381                         i  <= i_mas1;
382                         j  <= k_mas1;
383                         st <= s10;
384                     end if;
385                 else
386                     j  <= j_mas1;
387                     st <= s10;
388                 end if;
389             else
390                 st <= s16;
391             end if;
392
393         when s17 =>                             -- p = piv ; |p| = |piv| ; k = k + 1
394             p <= piv;
395             if piv(31) = '1' then
396                 pabs <= resultado(31 downto 0); -- |p| = 0 - piv
397             else
398                 pabs <= piv;
```

```
            end if;
            k <= k_mas1;
            if k_mas1 = n then                  -- k = n - 1 : ultimo paso
                st <= s18;
            else
                st <= s4;
            end if;

        when s18 =>                             -- det = +/- M(n,n) ; banco(n) = det
            det_bin <= det_nn;
            st      <= s19;

        ------------------------------------------------------------------
        -- Conversion XS3 (bloque aparte): primero det, luego el pivote
        ------------------------------------------------------------------
        when s19 =>                             -- inicio XS3 con |val|
            neg     <= val(31);
            dir_ant <= dir_p;
            st      <= s20;

        when s20 =>                             -- espera el XS3 y guarda el BCD
            if fin_xs3 = '1' then
                if conv_piv = '0' then
                    det_bcd   <= bcd_xs3;
                    det_neg_r <= neg;
                    conv_piv  <= '1';
                    st        <= s19;           -- ahora convierte el pivote
                else
                    piv_bcd   <= bcd_xs3;
                    piv_neg_r <= neg;
                    st        <= s21;
                end if;
            else
                st <= s20;
            end if;

        when s21 =>                             -- final = 1
            dira_r <= "00000";
            if inicio = '1' then
                st <= s1;                       -- nuevo calculo
            elsif dir_p /= dir_ant then
                st <= s19;                      -- cambio dir_p: convierte el nuevo pivote
            else
                st <= s21;
            end if;

        when others =>
            st <= s0;

    end case;
end if;
end process;

end Behavioral;
```

## src/bloque_division.vhd

```
----------------------------------------------------------------------------------
-- BLOQUE DE DIVISION POR RESTAS SUCESIVAS
--
-- Divide magnitudes: cociente = dividendo / divisor (division entera).
-- Tiene su propia unidad de control (ASM de 3 estados) y su propio camino de
-- datos: registro residuo R, registro divisor D, contador cociente Q y un
-- restador R - D. El signo del resultado lo aplica el ASM principal.
--
-- ASM:
--   d0 : espera. Si inicio = 1: R <- dividendo, D <- divisor, Q <- 0
--   d1 : si R - D >= 0: R <- R - D, Q <- Q + 1 (se queda en d1); si no, va a d2
--   d2 : final = 1 (un ciclo), vuelve a d0. El cociente queda en Q.
----------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.std_logic_unsigned.all;

entity bloque_division is
    Port ( clk, rst  : in  STD_LOGIC;
           inicio    : in  STD_LOGIC;
           dividendo : in  STD_LOGIC_VECTOR (49 downto 0);   -- magnitud, < 2^49
           divisor   : in  STD_LOGIC_VECTOR (31 downto 0);   -- magnitud, >= 1
           cociente  : out STD_LOGIC_VECTOR (31 downto 0);
           final     : out STD_LOGIC);
end bloque_division;

architecture Behavioral of bloque_division is

constant d0 : STD_LOGIC_VECTOR (1 downto 0) := "00";
constant d1 : STD_LOGIC_VECTOR (1 downto 0) := "01";
```

```
constant d2 : STD_LOGIC_VECTOR (1 downto 0) := "10";

constant CERO18 : STD_LOGIC_VECTOR (17 downto 0) := "000000000000000000";

signal st     : STD_LOGIC_VECTOR (1 downto 0);
signal R      : STD_LOGIC_VECTOR (49 downto 0);   -- residuo
signal D      : STD_LOGIC_VECTOR (31 downto 0);   -- divisor
signal Q      : STD_LOGIC_VECTOR (31 downto 0);   -- cociente
signal resta  : STD_LOGIC_VECTOR (49 downto 0);   -- R - D
signal q_mas1 : STD_LOGIC_VECTOR (31 downto 0);

begin

-- Camino de datos
resta  <= R - (CERO18 & D);           -- restador
q_mas1 <= Q + '1';                    -- contador del cociente

cociente <= Q;
final    <= '1' when st = d2 else '0';

-- ASM
process (clk, rst)
begin
if rst = '1' then
    st <= d0;
    R  <= (others => '0');
    D  <= (others => '0');
    Q  <= (others => '0');

elsif clk'event and clk = '1' then
    case st is

        when d0 =>                          -- espera
            if inicio = '1' then
                R  <= dividendo;
                D  <= divisor;
                Q  <= (others => '0');
                st <= d1;
            else
                st <= d0;
            end if;

        when d1 =>                          -- resta sucesiva
            if resta(49) = '0' then         -- R - D >= 0
                R  <= resta;
                Q  <= q_mas1;
                st <= d1;
            else
                st <= d2;
            end if;

        when d2 =>                          -- final = 1
            st <= d0;

        when others =>
            st <= d0;

    end case;
end if;
end process;

end Behavioral;
```

## src/xs3.vhd

```
----------------------------------------------------------------------------------
-- BLOQUE XS3: conversion binario -> BCD (sumar 3 y desplazar)
--
-- Tiene su propia unidad de control (ASM de 6 estados) y su propio camino de
-- datos: registro binario (32 bits), registro decimal (10 digitos = 40 bits),
-- un sumador de 4 bits (+3), un comparador de magnitud (> 4) y desplazadores.
-- Convierte una magnitud de 32 bits; el signo lo maneja el ASM principal.
--
-- ASM (un estado por cada paso del documento XS3):
--   x0 : espera. Si inicio = 1: bin <- bin_in
--   x1 : PASO 1. Registro decimal = 0; cont = 0
--   x2 : PASO 2. LSB del decimal = MSB del binario.
--        Si cont = 31 ya se hizo el ultimo paso 2 (PASO 5): va a x5
--   x3 : PASO 3. Revisa el digito d (0 a 9): si es > 4 le suma 3
--   x4 : PASO 4. Corrimiento a la izquierda de ambos registros; cont + 1
--   x5 : final = 1 (un ciclo), vuelve a x0. El resultado queda en bcd.
----------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.std_logic_unsigned.all;

entity xs3 is
```

```
    Port ( clk, rst : in  STD_LOGIC;
           inicio   : in  STD_LOGIC;
           bin_in   : in  STD_LOGIC_VECTOR (31 downto 0);
           bcd      : out STD_LOGIC_VECTOR (39 downto 0);
           final    : out STD_LOGIC);
end xs3;

architecture Behavioral of xs3 is

constant x0 : STD_LOGIC_VECTOR (2 downto 0) := "000";
constant x1 : STD_LOGIC_VECTOR (2 downto 0) := "001";
constant x2 : STD_LOGIC_VECTOR (2 downto 0) := "010";
constant x3 : STD_LOGIC_VECTOR (2 downto 0) := "011";
constant x4 : STD_LOGIC_VECTOR (2 downto 0) := "100";
constant x5 : STD_LOGIC_VECTOR (2 downto 0) := "101";

signal st      : STD_LOGIC_VECTOR (2 downto 0);
signal bin     : STD_LOGIC_VECTOR (31 downto 0);  -- registro binario
signal dec     : STD_LOGIC_VECTOR (39 downto 0);  -- registro decimal
signal cont    : STD_LOGIC_VECTOR (4 downto 0);   -- bits procesados (0..31)
signal d       : STD_LOGIC_VECTOR (3 downto 0);   -- digito revisado (0..9)
signal dig     : STD_LOGIC_VECTOR (3 downto 0);   -- digito d del registro decimal
signal dig_mas3 : STD_LOGIC_VECTOR (3 downto 0);
signal mayor4  : STD_LOGIC;
signal cont_mas1 : STD_LOGIC_VECTOR (4 downto 0);
signal d_mas1  : STD_LOGIC_VECTOR (3 downto 0);

begin

-- Camino de datos
with d select                                    -- multiplexor de digitos
    dig <= dec(3 downto 0)   when "0000",
           dec(7 downto 4)   when "0001",
           dec(11 downto 8)  when "0010",
           dec(15 downto 12) when "0011",
           dec(19 downto 16) when "0100",
           dec(23 downto 20) when "0101",
           dec(27 downto 24) when "0110",
           dec(31 downto 28) when "0111",
           dec(35 downto 32) when "1000",
           dec(39 downto 36) when "1001",
           "0000"            when others;

dig_mas3  <= dig + "0011";                       -- sumador de 4 bits
mayor4    <= '1' when dig > "0100" else '0';     -- comparador de magnitud
cont_mas1 <= cont + '1';
d_mas1    <= d + '1';

bcd   <= dec;
final <= '1' when st = x5 else '0';

-- ASM
process (clk, rst)
begin
if rst = '1' then
    st   <= x0;
    bin  <= (others => '0');
    dec  <= (others => '0');
    cont <= "00000";
    d    <= "0000";

elsif clk'event and clk = '1' then
    case st is

        when x0 =>                               -- espera
            if inicio = '1' then
                bin <= bin_in;
                st  <= x1;
            else
                st  <= x0;
            end if;

        when x1 =>                               -- PASO 1: registro decimal = 0
            dec  <= (others => '0');
            cont <= "00000";
            st   <= x2;

        when x2 =>                               -- PASO 2: LSB decimal = MSB binario
            dec(0) <= bin(31);
            if cont = "11111" then
                st <= x5;                        -- PASO 5: ultimo paso 2 hecho
            else
                d  <= "0000";
                st <= x3;
            end if;

        when x3 =>                               -- PASO 3: si digito(d) > 4, sumar 3
            if mayor4 = '1' then
```

```
                case d is
                    when "0000" => dec(3 downto 0)   <= dig_mas3;
                    when "0001" => dec(7 downto 4)   <= dig_mas3;
                    when "0010" => dec(11 downto 8)  <= dig_mas3;
                    when "0011" => dec(15 downto 12) <= dig_mas3;
                    when "0100" => dec(19 downto 16) <= dig_mas3;
                    when "0101" => dec(23 downto 20) <= dig_mas3;
                    when "0110" => dec(27 downto 24) <= dig_mas3;
                    when "0111" => dec(31 downto 28) <= dig_mas3;
                    when "1000" => dec(35 downto 32) <= dig_mas3;
                    when "1001" => dec(39 downto 36) <= dig_mas3;
                    when others => null;
                end case;
            end if;
            if d = "1001" then
                st <= x4;
            else
                d  <= d_mas1;
                st <= x3;
            end if;

        when x4 =>                              -- PASO 4: corrimiento a la izquierda
            dec  <= dec(38 downto 0) & '0';
            bin  <= bin(30 downto 0) & '0';
            cont <= cont_mas1;
            st   <= x2;

        when x5 =>                              -- final = 1
            st <= x0;

        when others =>
            st <= x0;

    end case;
end if;
end process;

end Behavioral;
```

## src/banco_pivotes.vhd

```
----------------------------------------------------------------------------------
-- BANCO DE REGISTROS DE PIVOTES (5 registros de 32 bits)
--
-- Registro k = pivote a(k,k) usado en el paso k (k = 1..n-1).
-- Registro n = determinante (el ultimo pivote es el determinante).
-- Escritura sincrona en la direccion dir_w; lectura combinacional en dir_r,
-- que viene de los switches (dir_p). Las direcciones 0, 6 y 7 leen 0.
----------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity banco_pivotes is
    Port ( clk, rst : in  STD_LOGIC;
           clr      : in  STD_LOGIC;                    -- borra los 5 registros
           we       : in  STD_LOGIC;
           dir_w    : in  STD_LOGIC_VECTOR (2 downto 0);
           dato     : in  STD_LOGIC_VECTOR (31 downto 0);
           dir_r    : in  STD_LOGIC_VECTOR (2 downto 0);
           q        : out STD_LOGIC_VECTOR (31 downto 0));
end banco_pivotes;

architecture Behavioral of banco_pivotes is

constant CERO32 : STD_LOGIC_VECTOR (31 downto 0) := x"00000000";

signal p1, p2, p3, p4, p5 : STD_LOGIC_VECTOR (31 downto 0);

begin

process (clk, rst)
begin
if rst = '1' then
    p1 <= CERO32;  p2 <= CERO32;  p3 <= CERO32;  p4 <= CERO32;  p5 <= CERO32;
elsif clk'event and clk = '1' then
    if clr = '1' then
        p1 <= CERO32;  p2 <= CERO32;  p3 <= CERO32;  p4 <= CERO32;  p5 <= CERO32;
    elsif we = '1' then
        case dir_w is
            when "001"  => p1 <= dato;
            when "010"  => p2 <= dato;
            when "011"  => p3 <= dato;
            when "100"  => p4 <= dato;
            when "101"  => p5 <= dato;
            when others => null;
        end case;
    end if;
```

```
end if;
end process;

with dir_r select
    q <= p1     when "001",
         p2     when "010",
         p3     when "011",
         p4     when "100",
         p5     when "101",
         CERO32 when others;

end Behavioral;
```

## src/sumador_restador.vhd

```
--------------------------------------------------------------------------------
-- SUMADOR/RESTADOR DE 50 bits (combinacional)
--
-- Unico sumador/restador del procesador; el ASM lo comparte entre estados
-- eligiendo sus operandos con multiplexores.
--   op = '0' : s = a + b
--   op = '1' : s = a + not(b) + 1 = a - b
--------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.std_logic_unsigned.all;

entity sumador_restador is
    Port ( a  : in  STD_LOGIC_VECTOR (49 downto 0);
           b  : in  STD_LOGIC_VECTOR (49 downto 0);
           op : in  STD_LOGIC;
           s  : out STD_LOGIC_VECTOR (49 downto 0));
end sumador_restador;

architecture Behavioral of sumador_restador is

signal bx : STD_LOGIC_VECTOR (49 downto 0);

begin

bx <= not b when op = '1' else b;
s  <= a + bx + op;

end Behavioral;
```

## src/multiplicador.vhd

```
--------------------------------------------------------------------------------
-- MULTIPLICADOR CON SIGNO 25 x 25 -> 50 bits (combinacional)
--
-- Unico multiplicador del procesador. Los puertos son std_logic_vector; el
-- paquete std_logic_signed hace que "*" los interprete en complemento a 2.
-- 25 bits alcanzan: en el paso k los operandos son determinantes de
-- submatrices de k x k (k <= 4), menores que 2^24 en valor absoluto.
--------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.std_logic_signed.all;

entity multiplicador is
    Port ( a : in  STD_LOGIC_VECTOR (24 downto 0);
           b : in  STD_LOGIC_VECTOR (24 downto 0);
           p : out STD_LOGIC_VECTOR (49 downto 0));
end multiplicador;

architecture Behavioral of multiplicador is
begin

p <= a * b;

end Behavioral;
```

## src/ram.vhd

```
--------------------------------------------------------------------------------
-- RAM DE TRABAJO (modelo de RAM del curso)
--
-- Es el modelo del aula con dos cambios de tamano:
--   * 64 palabras en lugar de 32, para que la direccion sea fila(3) & columna(3)
--     y no haya que multiplicar para ubicar a(i,j).
--   * Contenido inicial en cero (solo evita valores 'U' en la simulacion).
-- Escritura sincrona; lectura asincrona por una direccion aparte (rd_add).
-- Es la unica parte del diseno que usa un tipo distinto de std_logic (el
-- arreglo de la memoria) y conv_integer para la linea de direccion.
--------------------------------------------------------------------------------
```

```
12  library ieee;
13  use ieee.std_logic_1164.all;
14  use ieee.std_logic_unsigned.all;
15
16  entity RAM_memory is
17     port
18     (
19        c      : in  std_logic;
20        data   : in  std_logic_vector (31 downto 0);
21        wr_add : in  std_logic_vector (5 downto 0);
22        rd_add : in  std_logic_vector (5 downto 0);
23        we     : in  std_logic;
24        q      : out std_logic_vector (31 downto 0)
25     );
26  end RAM_memory;
27
28  architecture rtl of RAM_memory is
29     type mem is array(0 to 63) of std_logic_vector(31 downto 0);
30     signal ram_blk : mem := (others => (others => '0'));
31  begin
32
33     process (c, data, wr_add, rd_add, ram_blk)
34     begin
35        if (c'event and c = '1') then
36           if (we = '1') then
37              ram_blk(conv_integer(wr_add)) <= data;
38           end if;
39        end if;
40     end process;
41
42     q <= ram_blk(conv_integer(rd_add));
43
44  end rtl;
```

## src/rom_matriz.vhd

```
 1  ----------------------------------------------------------------------------------
 2  -- ROM DE LA MATRIZ (32 x 6 bits, lectura combinacional)
 3  --
 4  -- Hace parte de la arquitectura del procesador: para calcular el determinante
 5  -- de otra matriz se cambia este archivo y se vuelve a programar la FPGA.
 6  -- Archivo generado con: python3 sim/modelo_referencia.py rom ...
 7  --
 8  -- Posicion 00h = n; luego los elementos a(i,j) por columnas (a11, a21, ...),
 9  -- como en la Figura 1 del taller. Datos en complemento a 2 (-32 a 31).
10  --
11  -- Matriz (5x5):
12  --      4   -2    1    3    0
13  --     -2    6    2   -1    5
14  --      1    2   -7    4    2
15  --      3   -1    4    5   -3
16  --      0    5    2   -3    8
17  -- Resultado esperado: det = 362
18  -- Pivotes (dir_p = 1..5): 4, 20, -170, -639, 362
19  ----------------------------------------------------------------------------------
20  library IEEE;
21  use IEEE.STD_LOGIC_1164.ALL;
22
23  entity rom_matriz is
24      Port ( dira : in  STD_LOGIC_VECTOR (4 downto 0);
25             a    : out STD_LOGIC_VECTOR (5 downto 0));
26  end rom_matriz;
27
28  architecture Behavioral of rom_matriz is
29  begin
30
31  with dira select
32      a <= "000101" when "00000",   -- 00h: n = 5
33           "000100" when "00001",   -- 01h: a11 = 4
34           "111110" when "00010",   -- 02h: a21 = -2
35           "000001" when "00011",   -- 03h: a31 = 1
36           "000011" when "00100",   -- 04h: a41 = 3
37           "000000" when "00101",   -- 05h: a51 = 0
38           "111110" when "00110",   -- 06h: a12 = -2
39           "000110" when "00111",   -- 07h: a22 = 6
40           "000010" when "01000",   -- 08h: a32 = 2
41           "111111" when "01001",   -- 09h: a42 = -1
42           "000101" when "01010",   -- 0Ah: a52 = 5
43           "000001" when "01011",   -- 0Bh: a13 = 1
44           "000010" when "01100",   -- 0Ch: a23 = 2
45           "111001" when "01101",   -- 0Dh: a33 = -7
46           "000100" when "01110",   -- 0Eh: a43 = 4
47           "000010" when "01111",   -- 0Fh: a53 = 2
48           "000011" when "10000",   -- 10h: a14 = 3
49           "111111" when "10001",   -- 11h: a24 = -1
50           "000100" when "10010",   -- 12h: a34 = 4
51           "000101" when "10011",   -- 13h: a44 = 5
```

```
          "111101" when "10100",   -- 14h: a54 = -3
          "000000" when "10101",   -- 15h: a15 = 0
          "000101" when "10110",   -- 16h: a25 = 5
          "000010" when "10111",   -- 17h: a35 = 2
          "111101" when "11000",   -- 18h: a45 = -3
          "001000" when "11001",   -- 19h: a55 = 8
          "000000" when others;

end Behavioral;
```

## src/divisor_frecuencia.vhd

```
----------------------------------------------------------------------------------
-- DIVISOR DE FRECUENCIA (reloj de la Nexys A7 = 100 MHz, pin E3)
--
--   tick_1ms : pulso de un ciclo cada 1 ms (100 000 ciclos). Lo usan el
--              antirrebote (muestreo del boton) y el display (cambio de digito).
--   onda_1s  : onda cuadrada de periodo 1 s (cambia cada 500 ms).
--
-- Los pulsos se usan como habilitacion (clock enable): todo el diseno trabaja
-- con el mismo reloj de 100 MHz. El procesador no usa este divisor: con un
-- reloj de 1 s una matriz de 3x3 tardaria unos 13 minutos.
----------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.std_logic_unsigned.all;

entity divisor_frecuencia is
    Port ( clk, rst : in  STD_LOGIC;
           tick_1ms : out STD_LOGIC;
           onda_1s  : out STD_LOGIC);
end divisor_frecuencia;

architecture Behavioral of divisor_frecuencia is

constant FIN_1MS   : STD_LOGIC_VECTOR (16 downto 0) := "11000011010011111";   -- 99 999
constant FIN_500MS : STD_LOGIC_VECTOR (8 downto 0)  := "111110011";           -- 499

signal cuenta_ciclos : STD_LOGIC_VECTOR (16 downto 0);
signal cuenta_ms     : STD_LOGIC_VECTOR (8 downto 0);
signal tick          : STD_LOGIC;
signal onda          : STD_LOGIC;

begin

tick <= '1' when cuenta_ciclos = FIN_1MS else '0';

process (clk, rst)
begin
if rst = '1' then
    cuenta_ciclos <= (others => '0');
    cuenta_ms     <= (others => '0');
    onda          <= '0';
elsif clk'event and clk = '1' then
    if tick = '1' then
        cuenta_ciclos <= (others => '0');
        if cuenta_ms = FIN_500MS then
            cuenta_ms <= (others => '0');
            onda      <= not onda;
        else
            cuenta_ms <= cuenta_ms + '1';
        end if;
    else
        cuenta_ciclos <= cuenta_ciclos + '1';
    end if;
end if;
end process;

tick_1ms <= tick;
onda_1s  <= onda;

end Behavioral;
```

## src/antirrebote.vhd

```
----------------------------------------------------------------------------------
-- ANTIRREBOTE PARA EL BOTON DE INICIO
--
-- 1. Sincroniza el boton con dos flip-flops.
-- 2. Lo muestrea cada 1 ms (tick) en un registro de 8 muestras.
-- 3. El nivel estable cambia a '1' con 8 muestras en '1' (8 ms sin rebote) y
--    a '0' con 8 muestras en '0'.
-- 4. pulso = '1' durante un solo ciclo de reloj cuando el nivel estable pasa
--    de '0' a '1': una pulsacion genera un unico inicio.
----------------------------------------------------------------------------------
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;

entity antirrebote is
    Port ( clk, rst : in  STD_LOGIC;
           tick     : in  STD_LOGIC;      -- habilitacion de muestreo (1 ms)
           boton    : in  STD_LOGIC;
           pulso    : out STD_LOGIC);
end antirrebote;

architecture Behavioral of antirrebote is

signal b1, b2   : STD_LOGIC;
signal muestras : STD_LOGIC_VECTOR (7 downto 0);
signal estable, estable_ant : STD_LOGIC;

begin

process (clk, rst)
begin
if rst = '1' then
    b1 <= '0';  b2 <= '0';
    muestras    <= (others => '0');
    estable     <= '0';
    estable_ant <= '0';
elsif clk'event and clk = '1' then
    b1 <= boton;
    b2 <= b1;
    if tick = '1' then
        muestras <= muestras(6 downto 0) & b2;
    end if;
    if muestras = "11111111" then
        estable <= '1';
    elsif muestras = "00000000" then
        estable <= '0';
    end if;
    estable_ant <= estable;
end if;
end process;

pulso <= estable and not estable_ant;

end Behavioral;
```

## src/sincronizador.vhd

```
----------------------------------------------------------------------------------
-- SINCRONIZADOR DE LOS SWITCHES dir_p (dos flip-flops por bit)
-- Evita la metaestabilidad al pasar los switches al dominio del reloj.
----------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity sincronizador is
    Port ( clk, rst : in  STD_LOGIC;
           entrada  : in  STD_LOGIC_VECTOR (2 downto 0);
           salida   : out STD_LOGIC_VECTOR (2 downto 0));
end sincronizador;

architecture Behavioral of sincronizador is

signal s1, s2 : STD_LOGIC_VECTOR (2 downto 0);

begin

process (clk, rst)
begin
if rst = '1' then
    s1 <= "000";
    s2 <= "000";
elsif clk'event and clk = '1' then
    s1 <= entrada;
    s2 <= s1;
end if;
end process;

salida <= s2;

end Behavioral;
```

## src/display_8dig.vhd

```
----------------------------------------------------------------------------------
-- DISPLAY DE 8 DIGITOS MULTIPLEXADO (Nexys A7: anodo comun, activos en bajo)
--
-- AN7 = signo ('-' o apagado); AN6..AN0 = 7 digitos BCD.
-- Cambia de digito con cada tick (1 ms): barrido completo de 8 ms (125 Hz).
```

```
----------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.std_logic_unsigned.all;

entity display_8dig is
    Port ( clk, rst : in  STD_LOGIC;
           tick     : in  STD_LOGIC;
           bcd      : in  STD_LOGIC_VECTOR (27 downto 0);
           neg      : in  STD_LOGIC;
           an       : out STD_LOGIC_VECTOR (7 downto 0);
           seg      : out STD_LOGIC_VECTOR (6 downto 0);   -- seg(0)=CA ... seg(6)=CG
           dp       : out STD_LOGIC);
end display_8dig;

architecture Behavioral of display_8dig is

signal dsel      : STD_LOGIC_VECTOR (2 downto 0);   -- digito encendido
signal car       : STD_LOGIC_VECTOR (3 downto 0);   -- 0-9, A = '-', F = apagado
signal car_signo : STD_LOGIC_VECTOR (3 downto 0);

begin

process (clk, rst)
begin
if rst = '1' then
    dsel <= "000";
elsif clk'event and clk = '1' then
    if tick = '1' then
        dsel <= dsel + '1';
    end if;
end if;
end process;

car_signo <= "1010" when neg = '1' else "1111";

with dsel select
    car <= bcd(3 downto 0)   when "000",
           bcd(7 downto 4)   when "001",
           bcd(11 downto 8)  when "010",
           bcd(15 downto 12) when "011",
           bcd(19 downto 16) when "100",
           bcd(23 downto 20) when "101",
           bcd(27 downto 24) when "110",
           car_signo         when others;

with dsel select
    an <= "11111110" when "000",
          "11111101" when "001",
          "11111011" when "010",
          "11110111" when "011",
          "11101111" when "100",
          "11011111" when "101",
          "10111111" when "110",
          "01111111" when others;

with car select                                 --  gfedcba
    seg <= "1000000" when "0000",               --  0
           "1111001" when "0001",               --  1
           "0100100" when "0010",               --  2
           "0110000" when "0011",               --  3
           "0011001" when "0100",               --  4
           "0010010" when "0101",               --  5
           "0000010" when "0110",               --  6
           "1111000" when "0111",               --  7
           "0000000" when "1000",               --  8
           "0010000" when "1001",               --  9
           "0111111" when "1010",               --  -
           "1111111" when others;               --  apagado

dp <= '1';

end Behavioral;
```